第四周：**算法设计范式与 GPU 化**

分治、动态规划、贪心——及其并行化

优化算法课程

第 4 周

摘要

本周我们学习三大算法设计范式：分治、动态规划、贪心。但与传统算法课不同，我们重点关注一个新问题：**这些算法能否在 GPU 上高效并行？**传统算法假设”顺序执行”，而 GPU 提供数千个并行核心。

目录

1 三大范式概览 **4**

2 分治法 **4**

2.1 分治的基本结构 4

2.2 Master 定理 5

2.3 Master 定理例题 6

2.4 分治为什么天然并行? 10

3 分治经典：最近点对问题 **10**

3.1 问题定义 10

3.2 具体例子 10

3.3 分治算法 11

3.4 合并步骤的关键：带状区域 12

4 分治的 GPU 化：Merge Sort **15**

4.1 Merge Sort 回顾 15

4.2 GPU Merge Sort 的设计 17

4.3 单次合并的并行化 17

5 GPU 排序的最佳选择：基数排序 19
5.1 为什么基数排序适合 GPU? . . . 19
5.2 基数排序原理 . . . 20
5.3 计数排序：基数排序的基础 . . . 21
5.4 GPU 基数排序 . . . 22

6 分治的 GPU 化：FFT 24
6.1 FFT 的蝴蝶操作 . . . 24

7 动态规划 27
7.1 DP 的核心思想 . . . 27
7.2 证明最优子结构：剪切-粘贴法 . . . 27
7.3 例：编辑距离 . . . 28
7.4 例：最长公共子序列 (LCS) . . . 30
7.5 例：0-1 背包问题 . . . 33
7.5.1 0-1 背包的 FPTAS . . . 37
7.6 更多 DP 经典例子 . . . 38
7.6.1 例：爬楼梯（最基础线性 DP） . . . 38
7.6.2 例：最大子数组和 . . . 39
7.6.3 例：打家劫舍（线性 DP 带约束） . . . 40
7.6.4 例：Nim 取石子（博弈 DP） . . . 42
7.6.5 例：硬币找零 . . . 43
7.6.6 例：最长递增子序列 (LIS) . . . 44
7.6.7 例：矩阵链乘法 . . . 45
7.7 DP 的六种类型总结 . . . 47
7.8 状态压缩 DP：旅行商问题 . . . 48
7.9 树形 DP：最大独立集 . . . 50
7.10 DAG DP：最长路径 . . . 52

8 DP 的 GPU 困境与突破 54
8.1 为什么 DP 难以并行? . . . 54
8.2 对角线并行（Wavefront 并行） . . . 54

9 贪心算法 58
9.1 贪心的核心思想 . . . 58
9.2 例：活动选择问题 . . . 58
9.3 贪心正确性证明：交换论证 . . . 60
9.4 更多贪心经典例子 . . . 62
9.4.1 例：分数背包 . . . 62

9.4.2 例：跳跃游戏 64
9.4.3 例：加油站问题 65
9.4.4 例：最小化最大延迟（配对型交换） 66
9.4.5 例：霍夫曼编码 68
9.4.6 例：最小生成树（切割型交换） 69
9.5 贪心为什么难并行? 72

10 暴力的逆袭：当并行改变一切 73
10.1 核心思想 73

11 总结与实验 74

A 附录：CUDA Kernel 示例 74

1 三大范式概览

算法设计的核心问题

问题：给定一个计算问题，如何设计高效算法？
关键洞察：利用问题的**结构**来减少计算量。
三大范式代表了三种不同的”结构利用方式”：

范式	核心思想	结构类型	并行性
分治	分解 → 递归求解 → 合并	独立子问题	天然并行
动态规划	记录子问题答案，避免重复	重叠子问题	需要技巧
贪心	每步选局部最优	贪心选择性质	最难并行

直觉：为什么并行性不同？

分治：子问题**独立**，可以同时求解。
DP：子问题有**依赖**，但依赖结构有规律，可以找到并行的”波前”。
贪心：每一步依赖**前一步的结果**，顺序性最强。
但这不是绝对的——有时候放弃”聪明”的顺序算法，用”笨”的并行暴力反而更快！

2 分治法

2.1 分治的基本结构

分治法的三步

Divide：将问题分解为若干**规模更小**的子问题
Conquer：递归求解子问题（基本情况直接求解）
Combine：将子问题的解合并为原问题的解
递归式：设 $T(n)$ 为规模 n 的问题的时间复杂度

$$T(n) = a \cdot T(n/b) + f(n)$$

- a：子问题个数
- n/b：每个子问题的规模
- $f(n)$：分解与合并的代价

2.2 Master 定理

Master 定理（完整推导）

目标：求解递归式 $T(n) = a \cdot T(n/b) + f(n)$

展开递归：

$$\begin{aligned} T(n) &= a \cdot T(n/b) + f(n) \\ &= a\left[a \cdot T(n/b^2) + f(n/b)\right] + f(n) \\ &= a^2 T(n/b^2) + af(n/b) + f(n) \\ &= \cdots \\ &= a^k T(n/b^k) + \sum_{i=0}^{k-1} a^i f(n/b^i) \end{aligned}$$

当 $n/b^k = 1$，即 $k = \log_b n$ 时，递归到底：

$$T(n) = a^{\log_b n} T(1) + \sum_{i=0}^{\log_b n - 1} a^i f(n/b^i)$$

注意 $a^{\log_b n} = n^{\log_b a}$（取对数验证）。

关键比较：$n^{\log_b a}$ vs $f(n)$，谁增长更快？

核心结论：Master 定理的三种情况

设 $T(n) = aT(n/b) + f(n)$，令 $c^* = \log_b a$。

Case 1：若 $f(n) = O(n^{c^* - \epsilon})$（$f$ 增长慢）

$$T(n) = \Theta(n^{c^*}) = \Theta(n^{\log_b a})$$

递归调用主导。

Case 2：若 $f(n) = \Theta(n^{c^*} \log^k n)$（$f$ 和递归项平衡）

$$T(n) = \Theta(n^{c^*} \log^{k+1} n)$$

Case 3：若 $f(n) = \Omega(n^{c^* + \epsilon})$ 且 $af(n/b) \le cf(n)$（f 增长快）

$$T(n) = \Theta(f(n))$$

合并操作主导。

2.3 Master 定理例题

例 1：Merge Sort ——Case 2

递归式：$T(n) = 2T(n/2) + \Theta(n)$
参数识别：

- $a = 2$（分成 2 个子问题）
- $b = 2$（每个子问题规模减半）
- $f(n) = \Theta(n)$（合并需要线性时间）

计算临界指数：$c^* = \log_b a = \log_2 2 = 1$
比较：$f(n) = \Theta(n) = \Theta(n^1) = \Theta(n^{c^*})$
$\Rightarrow$ Case 2，$k = 0$
结论：$T(n) = \Theta(n^1 \cdot \log^{0+1} n) = \Theta(n \log n)$

例 2：二分搜索——Case 2

递归式：$T(n) = T(n/2) + \Theta(1)$
参数识别：

- $a = 1$（只递归一个子问题）
- $b = 2$（规模减半）
- $f(n) = \Theta(1)$（每层只做常数工作）

计算临界指数：$c^* = \log_2 1 = 0$
比较：$f(n) = \Theta(1) = \Theta(n^0) = \Theta(n^{c^*})$
$\Rightarrow$ Case 2，$k = 0$
结论：$T(n) = \Theta(n^0 \cdot \log n) = \Theta(\log n)$

例 3：Karatsuba 乘法——Case 1

问题：两个 n 位整数相乘
朴素方法：$O(n^2)$
Karatsuba 的分治：将 n 位数分成两半

$$x = x_1 \cdot 10^{n/2} + x_0, \quad y = y_1 \cdot 10^{n/2} + y_0$$

$$xy = x_1 y_1 \cdot 10^n + [(x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0] \cdot 10^{n/2} + x_0 y_0$$

只需要 **3 次**（而不是 4 次）$n/2$ 位乘法！

数值例子（12×34，$n = 2$，$n/2 = 1$）：

$$
\begin{aligned}
x = 12: &\quad x_1 = 1,\ x_0 = 2 \\
y = 34: &\quad y_1 = 3,\ y_0 = 4 \\
z_2 &= x_1 y_1 = 1 \times 3 = 3 \\
z_0 &= x_0 y_0 = 2 \times 4 = 8 \\
z_1 &= (x_1 + x_0)(y_1 + y_0) - z_2 - z_0 = 3 \times 7 - 3 - 8 = 21 - 11 = 10 \\
12 \times 34 &= 3 \times 10^2 + 10 \times 10^1 + 8 \times 10^0 = 300 + 100 + 8 = 408 \checkmark
\end{aligned}
$$

递归式：$T(n) = 3T(n/2) + \Theta(n)$
参数：$a = 3,\ b = 2,\ f(n) = \Theta(n)$
临界指数：$c^* = \log_2 3 \approx 1.585$
比较：$f(n) = \Theta(n) = O(n^{1.585-0.585})$
$\Rightarrow$ Case 1（f 增长慢于 n^{c^*}）
结论：$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
比朴素 $O(n^2)$ 快！

例 4：Strassen 矩阵乘法——Case 1

问题：两个 $n \times n$ 矩阵相乘
朴素方法：$O(n^3)$
朴素分治：分成 4 个 $n/2 \times n/2$ 子矩阵

$$
\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}
$$

需要 8 次矩阵乘法 $\Rightarrow T(n) = 8T(n/2) + \Theta(n^2)$
$c^* = \log_2 8 = 3$，所以 $T(n) = \Theta(n^3)$，没有改进！
Strassen 的技巧：通过巧妙组合，只需 **7 次**子矩阵乘法
定义 7 个中间乘积（其中 A, B, C, D, E, F, G, H 为原两矩阵的四块子矩阵）：

$$
\begin{aligned}
P_1 &= A(F - H) \\
P_2 &= (A + B)H \\
P_3 &= (C + D)E \\
P_4 &= D(G - E) \\
P_5 &= (A + D)(E + H) \\
P_6 &= (B - D)(G + H) \\
P_7 &= (A - C)(E + F)
\end{aligned}
$$

则结果矩阵的四块为：

$$
\begin{aligned}
AE + BG &= P_5 + P_4 - P_2 + P_6 \\
AF + BH &= P_1 + P_2 \\
CE + DG &= P_3 + P_4 \\
CF + DH &= P_5 + P_1 - P_3 - P_7
\end{aligned}
$$

数值验证（2×2 矩阵，各块退化为标量，$A = 1, B = 2, C = 3, D = 4$，$E = 5, F = 6, G = 7, H = 8$）：

朴素结果：$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$

Strassen 计算：

$$
\begin{aligned}
&P_1 = 1 \cdot (6 - 8) = -2, \quad P_2 = (1 + 2) \cdot 8 = 24, \quad P_3 = (3 + 4) \cdot 5 = 35 \\
&P_4 = 4 \cdot (7 - 5) = 8, \quad P_5 = (1 + 4)(5 + 8) = 65 \\
&P_6 = (2 - 4)(7 + 8) = -30, \quad P_7 = (1 - 3)(5 + 6) = -22
\end{aligned}
$$

$$
\begin{aligned}
&R_{11} = 65 + 8 - 24 - 30 = 19 \checkmark, \quad R_{12} = -2 + 24 = 22 \checkmark \\
&R_{21} = 35 + 8 = 43 \checkmark, \quad R_{22} = 65 + (-2) - 35 - (-22) = 50 \checkmark
\end{aligned}
$$

递归式：$T(n) = 7T(n/2) + \Theta(n^2)$

参数：$a = 7,\ b = 2,\ f(n) = \Theta(n^2)$

临界指数：$c^* = \log_2 7 \approx 2.807$

比较：$f(n) = \Theta(n^2) = O(n^{2.807-0.807})$

$\Rightarrow$ Case 1

结论：$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$

比朴素 $O(n^3)$ 快！

例 5：一个 Case 3 的例子

递归式：$T(n) = 2T(n/2) + \Theta(n^2)$

这对应什么算法？假设我们分治后，合并步骤需要检查所有 $O(n^2)$ 对元素。

参数：$a = 2,\ b = 2,\ f(n) = \Theta(n^2)$

临界指数：$c^* = \log_2 2 = 1$

比较：$f(n) = \Theta(n^2) = \Omega(n^{1+1})$

$\Rightarrow$ Case 3（f 增长快于 n^{c^*}）

验证正则条件：$af(n/b) = 2 \cdot (n/2)^2 = n^2/2 \leq cf(n)$（取 $c = 1/2$）

结论：$T(n) = \Theta(f(n)) = \Theta(n^2)$
合并操作主导，递归没有帮助！

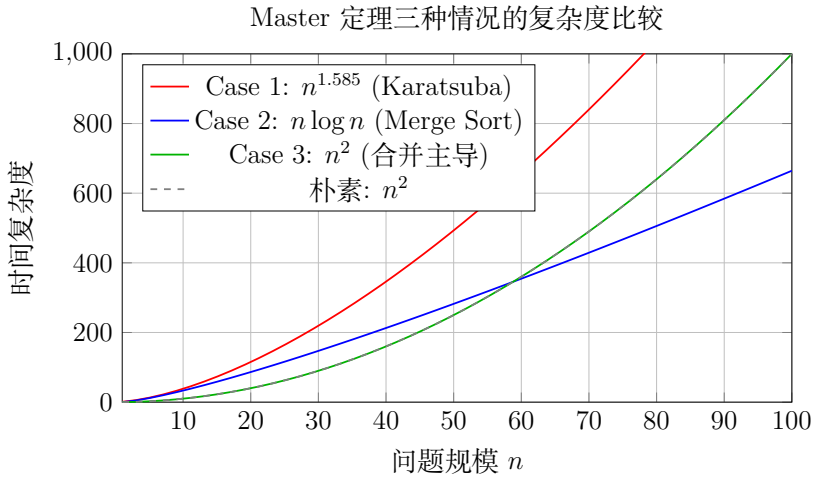


图 1: 不同分治策略的复杂度增长

核心结论：Master 定理速查表

算法	递归式	a	b	复杂度
二分搜索	$T(n) = T(n/2) + O(1)$	1	2	$O(\log n)$
Merge Sort	$T(n) = 2T(n/2) + O(n)$	2	2	$O(n \log n)$
Karatsuba	$T(n) = 3T(n/2) + O(n)$	3	2	$O(n^{1.585})$
Strassen	$T(n) = 7T(n/2) + O(n^2)$	7	2	$O(n^{2.807})$
朴素矩阵乘	$T(n) = 8T(n/2) + O(n^2)$	8	2	$O(n^3)$

记忆技巧：

- $c^* = \log_b a$ 是" 递归树叶子数" 的增长率
- $f(n)$ 是" 每层工作量" 的增长率
- 谁增长快，谁主导

2.4 分治为什么天然并行？

GPU 视角：分治的并行性

关键观察：分治产生的子问题是**独立的**！
例如 Merge Sort：

- 第 1 层：分成 2 个子数组，可以**并行**排序
- 第 2 层：4 个子数组，可以**并行**排序
- $\cdots$
- 第 k 层：2^k 个子数组，全部**并行**

并行时间复杂度（假设无限并行度）：
每层需要 $O(n/2^k)$ 时间做合并，共 $\log n$ 层：

$$T_{\text{parallel}}(n) = O(n) \quad \text{（瓶颈在最后一次合并）}$$

Speedup：$\frac{O(n\log n)}{O(n)} = O(\log n)$
但实际 GPU 有限并行度，需要更仔细的分析。

3 分治经典：最近点对问题

3.1 问题定义

最近点对问题

问题：给定平面上 n 个点 $P = \{p_1, p_2, \ldots, p_n\}$，其中 $p_i = (x_i, y_i)$。
求距离最近的两个点及其距离。
距离：欧几里得距离 $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
暴力算法：枚举所有 $\binom{n}{2}$ 对点，$O(n^2)$
目标：用分治做到 $O(n\log n)$

3.2 具体例子

例子：8 个点

输入：8 个点的坐标

点	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8
x	2	3	6	7	8	12	13	14
y	3	12	8	4	13	7	2	10

按 x 坐标排序：已经有序

分治：用 $x = 7.5$ 分成左右两半

- 左半部分 L：p_1, p_2, p_3, p_4
- 右半部分 R：p_5, p_6, p_7, p_8

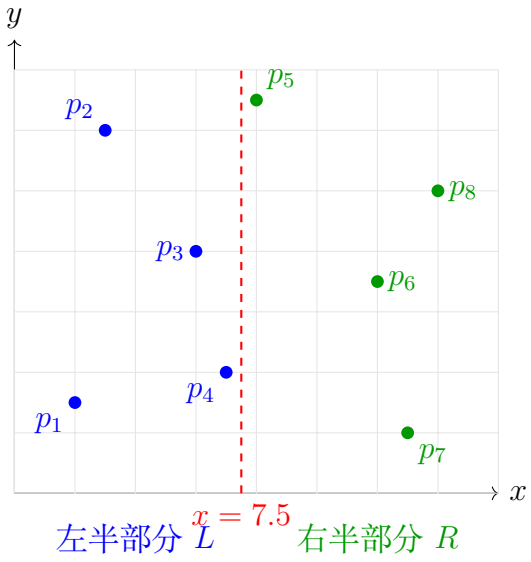


图 2: 最近点对：分治将点集一分为二

3.3 分治算法

分治策略

Divide：

1. 按 x 坐标排序（预处理，只做一次）
2. 用中位数 x_{mid} 将点集分成左右两半 L 和 R

Conquer：

1. 递归求 L 中的最近点对，距离为 δ_L
2. 递归求 R 中的最近点对，距离为 δ_R

Combine：

1. 令 $\delta = \min(\delta_L, \delta_R)$
2. **关键**：检查是否存在跨越分割线的点对，距离 $< \delta$
3. 返回全局最小距离

难点：合并步骤如何做到 $O(n)$?

手算例子（续）

Step 1：递归处理左半部分 $L = \{p_1, p_2, p_3, p_4\}$
左半部分的最近点对：计算所有距离

- $d(p_1, p_2) = \sqrt{(2-3)^2 + (3-12)^2} = \sqrt{1+81} = \sqrt{82} \approx 9.06$
- $d(p_1, p_3) = \sqrt{16+25} = \sqrt{41} \approx 6.40$
- $d(p_1, p_4) = \sqrt{25+1} = \sqrt{26} \approx 5.10$
- $d(p_2, p_3) = \sqrt{9+16} = \sqrt{25} = 5$
- $d(p_2, p_4) = \sqrt{16+64} = \sqrt{80} \approx 8.94$
- $d(p_3, p_4) = \sqrt{1+16} = \sqrt{17} \approx 4.12$

$\delta_L = \sqrt{17} \approx 4.12$（$p_3$ 和 p_4）
Step 2：递归处理右半部分 $R = \{p_5, p_6, p_7, p_8\}$
类似计算，$\delta_R = \sqrt{10} \approx 3.16$（$p_6$ 和 p_8）
Step 3：$\delta = \min(4.12, 3.16) = 3.16$

3.4 合并步骤的关键：带状区域

带状区域（Strip）

观察：跨越分割线的点对，两点都必须在分割线附近。
具体地：若 $d(p,q) < \delta$，则 $|x_p - x_{\text{mid}}| < \delta$ 且 $|x_q - x_{\text{mid}}| < \delta$
带状区域：$\text{Strip} = \{p : |x_p - x_{\text{mid}}| < \delta\}$
手算例子：$\delta = 3.16$，$x_{\text{mid}} = 7.5$
带状区域：$x \in (7.5 - 3.16, 7.5 + 3.16) = (4.34, 10.66)$
在带状区域内的点：$p_3(6,8)$, $p_4(7,4)$, $p_5(8,13)$

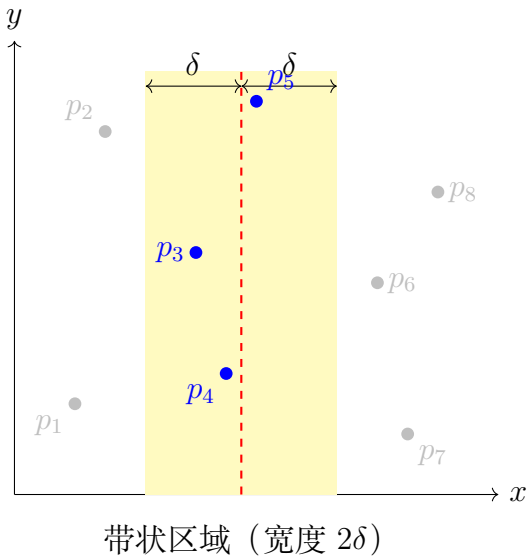


图 3: 带状区域：只有这里的点可能形成跨越分割线的更近点对

关键引理：带状区域中的检查是 $O(n)$

引理：将带状区域中的点按 y 坐标排序。对于每个点 p，只需检查 y 坐标在 $[y_p, y_p+\delta]$ 范围内的点。这样的点**最多 7 个**。

证明：

考虑一个 $\delta \times 2\delta$ 的矩形（宽 2δ，高 δ）。

将这个矩形分成 8 个 $\frac{\delta}{2} \times \frac{\delta}{2}$ 的小方格。

每个小方格的对角线长度 $= \frac{\delta}{\sqrt{2}} < \delta$。

关键：每个小方格内最多有 1 个点！

反证：若某小方格内有 2 个点 p, q，则 $d(p,q) < \delta$（对角线长度）。

但 p, q 都在同一侧（左或右），这与 δ 是该侧的最小距离矛盾。

结论：$\delta \times 2\delta$ 矩形内最多 8 个点，去掉 p 自己，最多检查 7 个。

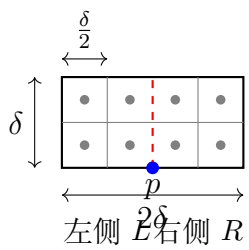


图 4: $\delta \times 2\delta$ 矩形分成 8 个小方格，每个方格最多 1 个点

完整算法

```
预处理：按 x 坐标排序得到 P_x，按 y 坐标排序得到 P_y

function ClosestPair(P_x, P_y):
if |P_x| ≤ 3 then
    return 暴力求解
end if
用中位数将 P_x 分成 L_x 和 R_x
相应地将 P_y 分成 L_y 和 R_y（保持 y 有序）
δ_L ← ClosestPair(L_x, L_y)
δ_R ← ClosestPair(R_x, R_y)
δ ← min(δ_L, δ_R)

{合并：检查带状区域}
Strip ← P_y 中满足 |x − x_mid| < δ 的点（保持 y 有序）
for i = 1 to |Strip| do
    for j = i + 1 to min(i + 7, |Strip|) do
        δ ← min(δ, d(Strip[i], Strip[j]))
    end for
end for
return δ
end function
```

复杂度：最近点对的复杂度分析

预处理：排序 $O(n \log n)$

递归式：

$$T(n) = 2T(n/2) + O(n)$$

- 分割：$O(n)$
- 构建带状区域：$O(n)$
- 检查带状区域：每个点检查 $O(1)$ 个点，共 $O(n)$

由 Master 定理：$a = 2$, $b = 2$, $f(n) = O(n)$

$c^* = \log_2 2 = 1 = f(n)$ 的指数 $\Rightarrow$ Case 2

$T(n) = O(n \log n)$

总复杂度：$O(n \log n)$

GPU 视角：最近点对的 GPU 化

并行性分析：

- 递归的两个子问题可以并行
- 带状区域检查：每个点独立检查，完全并行

GPU 策略：

- 当子问题足够小时，用暴力 $O(n^2)$（完全并行）
- 大问题用分治减少规模

对于小规模：$n < 10000$ 时，GPU 暴力可能比分治更快！
每个线程计算一对点的距离，n^2 个线程并行。

4 分治的 GPU 化：Merge Sort

4.1 Merge Sort 回顾

Merge Sort 的分治结构

输入：数组 $A[0..n-1]$
算法：

1. **Divide**：将数组分成两半 $A[0..n/2-1]$ 和 $A[n/2..n-1]$
2. **Conquer**：递归排序两半
3. **Combine**：合并两个有序数组

具体例子：排序 8 个数

输入：$A = [38, 27, 43, 3, 9, 82, 10, 71]$
分解过程（自顶向下）：

$$[38, 27, 43, 3, 9, 82, 10, 71]$$

$$[38, 27, 43, 3] \qquad [9, 82, 10, 71]$$

$$[38, 27] \quad [43, 3] \quad [9, 82] \quad [10, 71]$$

$$[38] \quad [27] \quad [43] \quad [3] \quad [9] \quad [82] \quad [10] \quad [71]$$

合并过程（自底向上）

第 1 轮合并：相邻两个元素合并

- $[38] + [27] \to [27, 38]$
- $[43] + [3] \to [3, 43]$
- $[9] + [82] \to [9, 82]$
- $[10] + [71] \to [10, 71]$

第 2 轮合并：

- $[27, 38] + [3, 43] \to [3, 27, 38, 43]$

  合并步骤：比较 27 和 3 → 取 3；比较 27 和 43 → 取 27；比较 38 和 43 → 取 38；取 43
- $[9, 82] + [10, 71] \to [9, 10, 71, 82]$

第 3 轮合并：

- $[3, 27, 38, 43] + [9, 10, 71, 82] \to [3, 9, 10, 27, 38, 43, 71, 82]$

输出：$[3, 9, 10, 27, 38, 43, 71, 82]$

Merge 操作详解

合并 $[3, 27, 38, 43]$ 和 $[9, 10, 71, 82]$：

步骤	左指针	右指针	输出	操作
1	3	9	3	$3 < 9$，取左，左指针右移
2	27	9	9	$27 > 9$，取右，右指针右移
3	27	10	10	$27 > 10$，取右，右指针右移
4	27	71	27	$27 < 71$，取左，左指针右移
5	38	71	38	$38 < 71$，取左，左指针右移
6	43	71	43	$43 < 71$，取左，左指针右移
7	–	71	71	左边空，取右
8	–	82	82	左边空，取右

结果：$[3, 9, 10, 27, 38, 43, 71, 82]$

4.2 GPU Merge Sort 的设计

GPU Merge Sort 的层级并行

思路：不用递归，改用**迭代**——从小到大合并。
算法：

1. 初始：每个元素是长度 1 的有序数组
2. 第 1 轮：相邻两个合并成长度 2 的有序数组（$n/2$ 次合并，**并行**）
3. 第 2 轮：相邻两个合并成长度 4 的有序数组（$n/4$ 次合并，**并行**）
4. $\cdots$
5. 第 $\log n$ 轮：两个 $n/2$ 的数组合并成完整有序数组

每一轮内部：所有合并操作**完全独立**，可以并行！

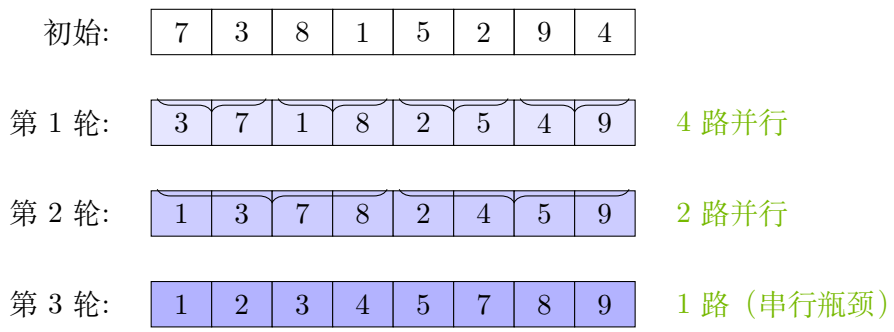


图 5: GPU Merge Sort：每轮内的合并操作并行执行

4.3 单次合并的并行化

并行合并：Merge 操作的 GPU 化

问题：合并两个有序数组 $A[0..m-1]$ 和 $B[0..k-1]$
串行方法：双指针，$O(m+k)$
并行方法：每个输出位置独立计算！
对于输出位置 i，需要确定 $C[i]$ 来自 A 还是 B。
关键洞察：$C[i]$ 是 $A \cup B$ 中第 i 小的元素。
设 $C[i] = A[j]$，则：

- A 中有 j 个元素 $\leq A[j]$
- B 中有 $i-j$ 个元素 $< A[j]$（用二分查找确定）

算法：对每个 $i \in \{0, \ldots, m+k-1\}$：

1. 二分搜索找到正确的 $(j, i-j)$ 分割
2. 确定 $C[i]$ 的值

复杂度：$O(\log(m+k))$ per thread，$(m+k)$ threads
总工作量 $O((m+k)\log(m+k))$，但并行时间 $O(\log(m+k))$！

实验：GPU Merge Sort

```python
import cupy as cp
import numpy as np
import time

def gpu_merge_sort_simple(arr):
    """简化版GPU Merge Sort：利用层级并行"""
    n = len(arr)
    d_arr = cp.asarray(arr)

    width = 1
    while width < n:
        # 每个block处理一对相邻的子数组
        # 这里用CuPy的argsort简化，实际应该写CUDA kernel
        for start in range(0, n, 2 * width):
            mid = min(start + width, n)
            end = min(start + 2 * width, n)
            if mid < end:
                merged = cp.concatenate([
                    d_arr[start:mid],
                    d_arr[mid:end]
                ])
                indices = cp.argsort(merged)
                d_arr[start:end] = merged[indices]
        width *= 2

    return cp.asnumpy(d_arr)

# 对比测试
n = 1_000_000
```

```python
arr = np.random.rand(n).astype(np.float32)

# CPU排序
start = time.time()
sorted_cpu = np.sort(arr.copy())
print(f"CPU numpy.sort: {time.time() - start:.3f}s")

# GPU排序(CuPy内置)
d_arr = cp.asarray(arr)
cp.cuda.Stream.null.synchronize()
start = time.time()
sorted_gpu = cp.sort(d_arr)
cp.cuda.Stream.null.synchronize()
print(f"GPU cupy.sort: {time.time() - start:.3f}s")
```

典型结果（$n = 10^6$）：

- CPU `numpy.sort`: 0.08s
- GPU `cupy.sort`: 0.01s
- 加速比：$\sim 8\times$

5 GPU 排序的最佳选择：基数排序

5.1 为什么基数排序适合 GPU？

比较排序 vs 非比较排序

比较排序（Merge Sort, Quick Sort）：

- 基于元素两两比较
- 下界 $\Omega(n \log n)$
- 依赖关系复杂，难以完全并行

非比较排序（Counting Sort, Radix Sort）：

- 利用元素的数值特性
- 可以达到 $O(n)$

- **天然适合并行！**

GPU 排序的实际选择：CUDA 库（Thrust, CUB）默认用**基数排序**！

5.2 基数排序原理

基数排序（Radix Sort）

核心思想：从最低位到最高位，逐位排序。每位使用**稳定排序**。

为什么要稳定？

稳定排序保持相等元素的相对顺序。

当高位相同时，低位的顺序已经排好，稳定排序不会打乱它！

算法：

```
for d = 0 to D − 1 do
    按第 d 位对数组进行稳定排序（通常用计数排序）
end for
```

复杂度：$O(D \cdot (n+k))$，其中 $D =$ 位数，$k =$ 每位的取值范围

对于 32 位整数，$D = 32$，若按 8 位分组则 $D = 4$，$k = 256$

基数排序手算例子

输入：$[329, 457, 657, 839, 436, 720, 355]$

按**十进制**每位排序（$k = 10$）：

第 1 轮：按个位排序

个位	数字
0	720
5	355
6	436
7	457, 657
9	329, 839

结果：$[720, 355, 436, 457, 657, 329, 839]$

第 2 轮：按十位排序（保持稳定性！）

十位	数字
2	720, 329
3	436, 839
5	355, 457, 657

结果：$[720, 329, 436, 839, 355, 457, 657]$

第 3 轮：按百位排序

百位	数字
3	329, 355
4	436, 457
6	657
7	720
8	839

结果：$[329, 355, 436, 457, 657, 720, 839]$ 排序完成！

稳定性的关键作用

观察第 3 轮：329 和 355 的百位都是 3
为什么 329 在 355 前面？
因为第 2 轮后，329 在 355 前面（十位 $2 < 5$）
第 3 轮的稳定排序**保持了这个顺序**！
反例：如果第 3 轮用不稳定排序，可能得到 $[355, 329, \ldots]$，错误！

5.3 计数排序：基数排序的基础

计数排序（Counting Sort）

问题：排序 n 个元素，每个元素的值在 $[0, k-1]$ 范围内
算法：

```
{Step 1: 计数}
for i = 0 to n − 1 do
    count[A[i]] ← count[A[i]] + 1
end for

{Step 2: 前缀和（确定每个值的起始位置）}
for i = 1 to k − 1 do
    count[i] ← count[i] + count[i − 1]
end for

{Step 3: 放置元素（从后往前保证稳定性）}
for i = n − 1 down to 0 do
    count[A[i]] ← count[A[i]] − 1
    output[count[A[i]]] ← A[i]
```

15: **end for**

复杂度：$O(n+k)$

计数排序手算

输入：$A=[4,2,2,8,3,3,1]$，$k=9$

Step 1: 计数

值	0	1	2	3	4	5	6	7	8
count	0	1	2	2	1	0	0	0	1

Step 2: 前缀和（count[i] = 值 $\leq i$ 的元素个数）

值	0	1	2	3	4	5	6	7	8
count	0	1	3	5	6	6	6	6	7

Step 3: 放置（从后往前）

- $A[6]=1$: $count[1]=1\to 0$, $output[0]=1$
- $A[5]=3$: $count[3]=5\to 4$, $output[4]=3$
- $A[4]=3$: $count[3]=4\to 3$, $output[3]=3$
- $A[3]=8$: $count[8]=7\to 6$, $output[6]=8$
- $A[2]=2$: $count[2]=3\to 2$, $output[2]=2$
- $A[1]=2$: $count[2]=2\to 1$, $output[1]=2$
- $A[0]=4$: $count[4]=6\to 5$, $output[5]=4$

输出：$[1,2,2,3,3,4,8]$

5.4 GPU 基数排序

GPU 视角：基数排序的 GPU 并行性

每一轮（按某一位排序）的三个步骤：

Step 1: 计数 → 完全并行

- 每个线程统计自己负责的元素
- 使用原子操作更新全局计数
- 或：先局部计数，再归约

Step 2: 前缀和 → 高度并行

- 经典并行算法：**Blelloch scan**，$O(\log k)$ 并行时间
- **上扫（Upsweep / Reduce）阶段**：构建规约树，从叶向根，每层相邻元素两两相加，$\lceil \log_2 k \rceil$ 轮后根节点保存总和
- **下扫（Downsweep）阶段**：将根节点置零，从根向叶，每层节点将自身值传给左孩子，并将左孩子原值加上自身值后传给右孩子，$\lceil \log_2 k \rceil$ 轮后各叶节点得到前缀和
- 两阶段共 $O(\log k)$ 轮，每轮 $O(k)$ 次加法 $\Rightarrow$ 总工作量 $O(k \log k)$，并行时间 $O(\log k)$

Step 3: 放置 → 完全并行

- 每个元素独立计算自己的目标位置
- 使用原子操作避免冲突

总并行复杂度：$O(D \cdot \log n)$，其中 $D =$ 轮数

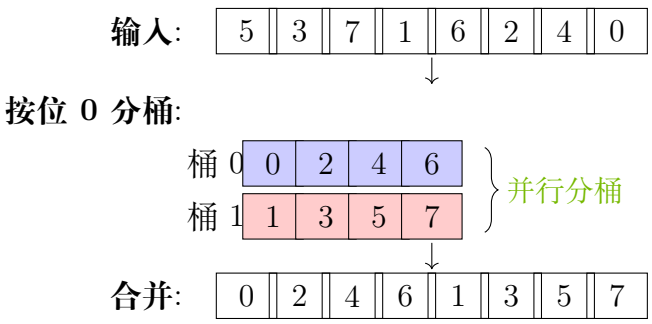


图 6: 基数排序第一轮：按最低位分成偶数桶和奇数桶

GPU 基数排序的优化技巧

1. 按多位分组

- 不按 1 位，而按 4 位或 8 位分组
- 减少轮数：32 位整数只需 4 轮（8 位/轮）
- 桶数增加：$2^8 = 256$ 个桶

2. 局部排序 + 全局合并

- 每个线程块先对局部数据排序
- 再合并各块的结果
- 减少全局内存访问

3. 避免原子操作冲突

- 使用线程块级别的共享内存计数
- 块内归约后再更新全局计数

复杂度：排序算法 GPU 性能对比

算法	串行复杂度	并行复杂度	GPU 适合度
Merge Sort	$O(n \log n)$	$O(\log^2 n)$	
Quick Sort	$O(n \log n)$	$O(\log^2 n)$ 期望	
Bitonic Sort	$O(n \log^2 n)$	$O(\log^2 n)$	
Radix Sort	$O(Dn)$	$O(D \log n)$	

实际性能（NVIDIA GPU，$n = 10^8$ 个 32 位整数）：

- Thrust radix sort: ~ 50 ms
- Thrust merge sort: ~ 200 ms
- CPU std::sort: ~ 8000 ms

基数排序比并行归并排序快 4×，比 CPU 快 160×！

6 分治的 GPU 化：FFT

6.1 FFT 的蝴蝶操作

Cooley-Tukey FFT 回顾

DFT 定义：给定 $x_0, \ldots, x_{n-1}$，计算

$$X_k = \sum_{j=0}^{n-1} x_j \cdot e^{-2\pi ijk/n}, \quad k = 0, \ldots, n-1$$

直接计算：$O(n^2)$

FFT 的分治思想：将 DFT 分解为奇偶两部分

设 $\omega_n = e^{-2\pi i/n}$，则：

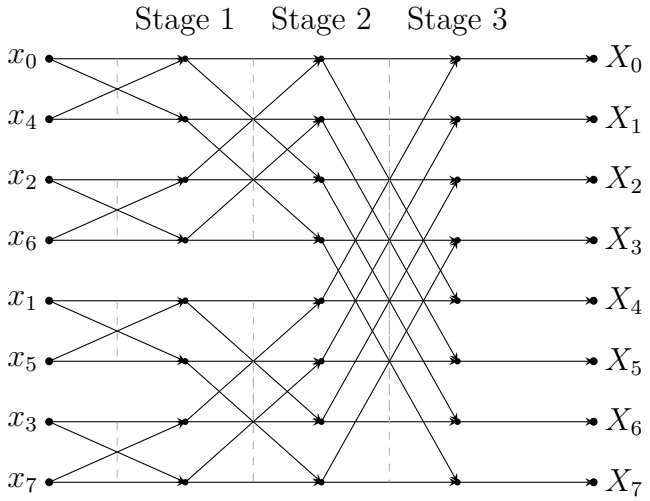
$$
\begin{aligned}
X_k &= \sum_{j=0}^{n/2-1} x_{2j}\omega_n^{2jk} + \sum_{j=0}^{n/2-1} x_{2j+1}\omega_n^{(2j+1)k} \\
&= \underbrace{\sum_{j=0}^{n/2-1} x_{2j}\omega_{n/2}^{jk}}_{E_k} + \omega_n^k \underbrace{\sum_{j=0}^{n/2-1} x_{2j+1}\omega_{n/2}^{jk}}_{O_k}
\end{aligned}
$$

其中 E_k 是偶数项的 $n/2$ 点 DFT，O_k 是奇数项的 $n/2$ 点 DFT。

蝴蝶操作：

$$X_k = E_k + \omega_n^k O_k, \quad X_{k+n/2} = E_k - \omega_n^k O_k$$

8 点 radix-2 DIT FFT 的蝴蝶结构



输入按 bit-reversal 顺序排列
$\{x_0, x_4, x_2, x_6, x_1, x_5, x_3, x_7\}$

输出为自然顺序
$\{X_0, \ldots, X_7\}$

图 7: 8 点 radix-2 DIT FFT 的蝴蝶结构示意图。每个 stage 内的 butterfly 单元彼此独立，因此可并行执行；但不同 stage 之间存在顺序依赖。图中输入采用 bit-reversal 顺序，因此输出为自然顺序。

4 点 FFT 数值例子

输入：$x = [1, 2, 3, 4]$

目标：计算 DFT $X_k = \sum_{j=0}^{3} x_j \cdot e^{-2\pi ijk/4}$

旋转因子：$\omega_4 = e^{-2\pi i/4} = e^{-\pi i/2} = -i$

所以：$\omega_4^0 = 1$，$\omega_4^1 = -i$，$\omega_4^2 = -1$，$\omega_4^3 = i$

Bit-reversal 排列：

原索引	二进制	反转后
0	00	$00 \to 0$
1	01	$10 \to 2$
2	10	$01 \to 1$
3	11	$11 \to 3$

重排后：$[x_0, x_2, x_1, x_3] = [1, 3, 2, 4]$

Stage 1：2 点 DFT（两组）

- 蝴蝶 1：$[1, 3] \to [1 + 3, 1 - 3] = [4, -2]$
- 蝴蝶 2：$[2, 4] \to [2 + 4, 2 - 4] = [6, -2]$

结果：$[4, -2, 6, -2]$

Stage 2：4 点 DFT

- $X_0 = 4 + \omega_4^0 \cdot 6 = 4 + 6 = 10$
- $X_1 = -2 + \omega_4^1 \cdot (-2) = -2 + (-i)(-2) = -2 + 2i$
- $X_2 = 4 + \omega_4^2 \cdot 6 = 4 - 6 = -2$
- $X_3 = -2 + \omega_4^3 \cdot (-2) = -2 + i(-2) = -2 - 2i$

输出：$X = [10, -2 + 2i, -2, -2 - 2i]$

验证：$X_0 = 1 + 2 + 3 + 4 = 10$ （直流分量 = 求和）

GPU 视角：FFT 的 GPU 并行性

结构：$\log_2 n$ 个 stages，每个 stage 有 $n/2$ 个蝴蝶操作。

并行性：

- 同一 stage 内：所有蝴蝶操作**完全独立**
- 不同 stage：必须**顺序**执行（后一个依赖前一个）

GPU 实现要点：

- 每个线程处理一个蝴蝶操作
- Stage 之间用 `__syncthreads()` 同步
- 利用 shared memory 减少 global memory 访问

复杂度：

- 串行：$O(n \log n)$
- 并行（$n/2$ 个处理器）：$O(\log n)$

7 动态规划

7.1 DP 的核心思想

动态规划的两个关键性质

性质 1：最优子结构
问题的最优解**包含**子问题的最优解。
例：最短路径——$s \to t$ 的最短路径经过 v，则 $s \to v$ 部分也是最短路径。
性质 2：重叠子问题
不同的递归分支会求解**相同**的子问题。
例：Fibonacci $F(n) = F(n-1) + F(n-2)$，朴素递归重复计算 $F(k)$ 指数次。
DP 的做法：

1. 定义状态（子问题的参数化）
2. 写出状态转移方程（如何从子问题构造原问题）
3. 确定计算顺序（保证子问题先于原问题求解）
4. 用表格存储结果，避免重复计算

7.2 证明最优子结构：剪切-粘贴法

最短路径的最优子结构证明

命题：设 $p = v_0 \to v_1 \to \cdots \to v_k$ 是从 v_0 到 v_k 的最短路径。则对任意 $0 \le i < j \le k$，子路径 $p_{ij} = v_i \to v_{i+1} \to \cdots \to v_j$ 是从 v_i 到 v_j 的最短路径。
证明（剪切-粘贴法/反证法）：
假设 p_{ij} 不是 v_i 到 v_j 的最短路径。
则存在更短的路径 p'_{ij}，满足 $w(p'_{ij}) < w(p_{ij})$。
剪切：从 p 中移除 p_{ij}
粘贴：用 p'_{ij} 替换
得到新路径 $p' = v_0 \to \cdots \to v_i \xrightarrow{p'_{ij}} v_j \to \cdots \to v_k$

则 $w(p') = w(p) - w(p_{ij}) + w(p'_{ij}) < w(p)$
这与 p 是最短路径矛盾！
结论：p_{ij} 必须是 v_i 到 v_j 的最短路径。□

直觉：剪切-粘贴法的通用模式

目标：证明最优解包含子问题的最优解
步骤：

1. 假设最优解中的某个子问题解不是最优的
2. 用那个子问题的真正最优解" 替换" 它
3. 证明替换后得到更好的解
4. 与原解是最优矛盾

这个方法几乎适用于所有 DP 问题的最优子结构证明。

7.3 例：编辑距离

编辑距离（Levenshtein Distance）

问题：给定两个字符串 $A[1..m]$ 和 $B[1..n]$，求将 A 变成 B 的最少操作数。
允许的操作：插入、删除、替换（各计 1 次）。
状态定义：$dp[i][j]$ = 将 $A[1..i]$ 变成 $B[1..j]$ 的最少操作数
状态转移：

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 & \text{（删除 } A[i]\text{）} \\ dp[i][j-1] + 1 & \text{（插入 } B[j]\text{）} \\ dp[i-1][j-1] + \mathbf{1}_{A[i] \neq B[j]} & \text{（匹配或替换）} \end{cases}$$

边界条件：

- $dp[0][j] = j$（空串变成 $B[1..j]$ 需要 j 次插入）
- $dp[i][0] = i$（$A[1..i]$ 变成空串需要 i 次删除）

计算顺序：从左到右、从上到下填表
时间复杂度：$O(mn)$，空间复杂度：$O(mn)$（可优化到 $O(\min(m,n))$）

编辑距离具体例子：手算填表

输入：$A =$ "CAT"，$B =$ "CAR"
将"CAT" 变成"CAR" 需要多少次操作?
填表过程：

$dp[i][j]$	ϵ	C	A	R
ϵ	0	1	2	3
C	1	0	1	2
A	2	1	0	1
T	3	2	1	1

逐格计算：
$dp[1][1]$ ($A[1]$=C, $B[1]$=C)：字符相同！

$$dp[1][1] = \min(dp[0][1]+1, dp[1][0]+1, dp[0][0]+0) = \min(2,2,0) = 0$$

$dp[1][2]$ ($A[1]$=C, $B[2]$=A)：字符不同

$$dp[1][2] = \min(dp[0][2]+1, dp[1][1]+1, dp[0][1]+1) = \min(3,1,2) = 1$$

$dp[2][2]$ ($A[2]$=A, $B[2]$=A)：字符相同！

$$dp[2][2] = \min(dp[1][2]+1, dp[2][1]+1, dp[1][1]+0) = \min(2,2,0) = 0$$

$dp[3][3]$ ($A[3]$=T, $B[3]$=R)：字符不同

$$dp[3][3] = \min(dp[2][3]+1, dp[3][2]+1, dp[2][2]+1) = \min(2,2,1) = 1$$

答案：$dp[3][3] = 1$（把 T 替换成 R）

编辑距离状态转移的证明

命题：状态转移方程正确，即

$$dp[i][j] = \min \begin{cases} dp[i-1][j]+1 \\ dp[i][j-1]+1 \\ dp[i-1][j-1] + \mathbf{1}_{A[i]\neq B[j]} \end{cases}$$

证明：
设 S 是将 $A[1..i]$ 变成 $B[1..j]$ 的任意最优操作序列。
考虑 S 的**最后一步操作**对 $A[i]$ 做了什么：
Case 1：最后删除了 $A[i]$

则 S 的前面部分将 $A[1..i-1]$ 变成 $B[1..j]$
$\Rightarrow |S| = dp[i-1][j] + 1$
（如果前面部分不是最优，可以剪切-粘贴得到矛盾）
Case 2：最后插入了 $B[j]$
则 S 的前面部分将 $A[1..i]$ 变成 $B[1..j-1]$
$\Rightarrow |S| = dp[i][j-1] + 1$
Case 3：$A[i]$ 和 $B[j]$ 匹配（或被替换）
则 S 的前面部分将 $A[1..i-1]$ 变成 $B[1..j-1]$
$\Rightarrow |S| = dp[i-1][j-1] + \mathbf{1}_{A[i]\neq B[j]}$
结论：最优解必落入三种情况之一，取最小值即可。□

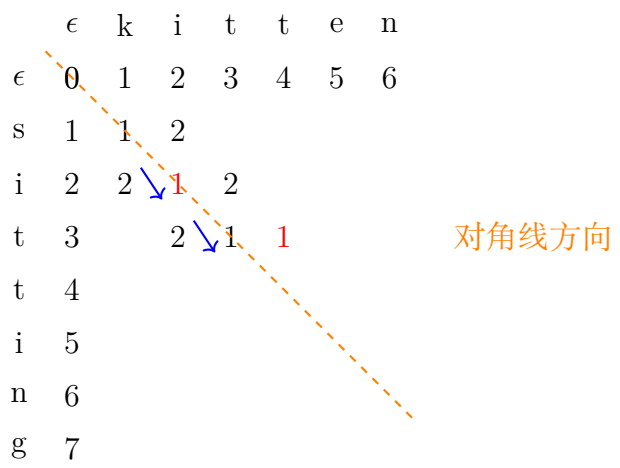


图 8: 编辑距离 DP 表格：每个格子依赖左、上、左上三个格子

7.4 例：最长公共子序列 (LCS)

LCS 问题定义

问题：给定两个序列 $X = x_1x_2\cdots x_m$ 和 $Y = y_1y_2\cdots y_n$，求最长公共子序列的长度。
子序列：从原序列中删除若干元素（可以不连续）得到的序列。
例：$X = \text{ABCBDAB}$，$Y = \text{BDCABA}$
LCS = BCBA 或 BDAB，长度 = 4
状态定义：$dp[i][j] = X[1..i]$ 和 $Y[1..j]$ 的 LCS 长度

LCS 最优子结构的证明

命题：设 $Z = z_1 z_2 \cdots z_k$ 是 $X[1..m]$ 和 $Y[1..n]$ 的一个 LCS。

1. 若 $x_m = y_n$，则 $z_k = x_m = y_n$，且 $Z[1..k-1]$ 是 $X[1..m-1]$ 和 $Y[1..n-1]$ 的 LCS
2. 若 $x_m \neq y_n$ 且 $z_k \neq x_m$，则 Z 是 $X[1..m-1]$ 和 $Y[1..n]$ 的 LCS
3. 若 $x_m \neq y_n$ 且 $z_k \neq y_n$，则 Z 是 $X[1..m]$ 和 $Y[1..n-1]$ 的 LCS

证明（情况 1）：
假设 $x_m = y_n$。
Step 1：证明 $z_k = x_m = y_n$
反证：若 $z_k \neq x_m$，则可以将 x_m 附加到 Z 末尾，得到更长的公共子序列，矛盾。
Step 2：证明 $Z[1..k-1]$ 是 $X[1..m-1]$ 和 $Y[1..n-1]$ 的 LCS
显然 $Z[1..k-1]$ 是 $X[1..m-1]$ 和 $Y[1..n-1]$ 的公共子序列。
若存在更长的公共子序列 W，长度 $> k-1$，则 W 后接 x_m 是 X 和 Y 的公共子序列，长度 $> k$，与 Z 是 LCS 矛盾。□

LCS 状态转移方程

由最优子结构，状态转移方程为：

$$dp[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ dp[i-1][j-1] + 1 & \text{if } x_i = y_j \\ \max(dp[i-1][j], dp[i][j-1]) & \text{if } x_i \neq y_j \end{cases}$$

正确性证明（归纳法）：
归纳假设：对所有 $i' < i$ 或 $j' < j$，$dp[i'][j']$ 正确。
归纳步骤：
Case $x_i = y_j$：
由情况 1，LCS 以 $x_i = y_j$ 结尾，去掉后是 $X[1..i-1]$ 和 $Y[1..j-1]$ 的 LCS。
由归纳假设，$dp[i-1][j-1]$ 正确，故 $dp[i][j] = dp[i-1][j-1] + 1$。
Case $x_i \neq y_j$：
LCS 要么不包含 x_i（情况 2），要么不包含 y_j（情况 3）。
由归纳假设，$dp[i-1][j]$ 和 $dp[i][j-1]$ 正确，取最大值即可。□

LCS 具体例子：手算填表

输入：$X = \text{ABCBDAB}$（长度 7），$Y = \text{BDCABA}$（长度 6）

填表过程（展示关键步骤）：

初始化：第 0 行和第 0 列全为 0

填 $dp[1][*]$（$X[1] = A$）：

- $dp[1][1]$：$A \neq B$，$\max(dp[0][1], dp[1][0]) = \max(0, 0) = 0$
- $dp[1][2]$：$A \neq D$，$\max(dp[0][2], dp[1][1]) = \max(0, 0) = 0$
- $dp[1][3]$：$A \neq C$，$\max(dp[0][3], dp[1][2]) = \max(0, 0) = 0$
- $dp[1][4]$：$A = A$（$Y[4] = A$），$dp[0][3] + 1 = 0 + 1 = 1$
- $dp[1][5]$：$A \neq B$（$Y[5] = B$），$\max(dp[0][5], dp[1][4]) = \max(0, 1) = 1$
- $dp[1][6]$：$A = A$（$Y[6] = A$），$dp[0][5] + 1 = 0 + 1 = 1$

填 $dp[2][*]$（$X[2] = B$）：

- $dp[2][1]$：$B = B$（$Y[1] = B$），$dp[1][0] + 1 = 0 + 1 = 1$
- $dp[2][2]$：$B \neq D$，$\max(dp[1][2], dp[2][1]) = \max(0, 1) = 1$
- $dp[2][3]$：$B \neq C$，$\max(dp[1][3], dp[2][2]) = \max(0, 1) = 1$
- $dp[2][4]$：$B \neq A$，$\max(dp[1][4], dp[2][3]) = \max(1, 1) = 1$
- $dp[2][5]$：$B = B$（$Y[5] = B$），$dp[1][4] + 1 = 1 + 1 = 2$
- $dp[2][6]$：$B \neq A$，$\max(dp[1][6], dp[2][5]) = \max(1, 2) = 2$

$i \backslash j$	0	B	D	C	A	B	A
0	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	$\boxed{4}$

图 9: LCS 表格：$dp[7][6] = 4$。回溯路径给出 LCS = BCBA（或 BDAB）

LCS 回溯找具体序列

从 $dp[m][n]$ 开始回溯：

```
i ← m, j ← n, result ← 空串
while i > 0 and j > 0 do
    if X[i] = Y[j] then
        result ← X[i] + result（前置）
        i ← i − 1, j ← j − 1
    else if dp[i − 1][j] > dp[i][j − 1] then
        i ← i − 1
    else
        j ← j − 1
    end if
end while
return result
```

例子中的回溯：

- $(7,6)$：$X[7] = B \neq Y[6] = A$，$dp[6][6] = 4 > dp[7][5] = 4$？否，$j \leftarrow 5$
- $(7,5)$：$X[7] = B = Y[5] = B$，输出 B，$(6,4)$
- $(6,4)$：$X[6] = A = Y[4] = A$，输出 A，$(5,3)$
- $(5,3)$：$X[5] = D \neq Y[3] = C$，$dp[4][3] = 2 = dp[5][2] = 2$，$j \leftarrow 2$
- $(5,2)$：$X[5] = D = Y[2] = D$，输出 D，$(4,1)$
- $(4,1)$：$X[4] = B = Y[1] = B$，输出 B，$(3,0)$
- 结束

结果：LCS = BDAB（从后往前：B, A, D, B）

7.5 例：0-1 背包问题

0-1 背包问题

问题：n 个物品，第 i 个重量 w_i，价值 v_i。背包容量 W。选择物品使总价值最大，且总重量 $\leq W$。

状态定义：$dp[i][j]$ = 前 i 个物品、容量为 j 时的最大价值

状态转移：

$$dp[i][j] = \begin{cases} dp[i-1][j] & \text{if } w_i > j(\text{放不下}) \\ \max(dp[i-1][j], dp[i-1][j-w_i] + v_i) & \text{otherwise} \end{cases}$$

解释：

- $dp[i-1][j]$：不选第 i 个物品
- $dp[i-1][j-w_i] + v_i$：选第 i 个物品（前提是放得下）

0-1 背包的完整算法

输入：物品重量 $w[1..n]$，价值 $v[1..n]$，背包容量 W
输出：最大价值

```
初始化：dp[0][j] ← 0 for all j = 0, 1, ..., W
for i = 1 to n do
    for j = 0 to W do
        if w[i] > j then
            dp[i][j] ← dp[i−1][j] {放不下，不选}
        else
            dp[i][j] ← max(dp[i−1][j], dp[i−1][j−w[i]] + v[i])
        end if
    end for
end for
return dp[n][W]
```

时间复杂度：$O(nW)$（伪多项式时间）
空间复杂度：$O(nW)$，可优化到 $O(W)$

具体例子：手算填表

输入：4 个物品，背包容量 $W = 5$

物品 i	1	2	3	4
重量 w_i	2	1	3	2
价值 v_i	3	2	4	3

填表过程：
初始化：$dp[0][j] = 0$（没有物品可选）
$i = 1$（物品 1：重量 2，价值 3）：

- $j = 0, 1$：放不下，$dp[1][j] = dp[0][j] = 0$
- $j = 2, 3, 4, 5$：$dp[1][j] = \max(0, dp[0][j-2] + 3) = 3$

$i = 2$（物品 2：重量 1，价值 2）：

- $j = 0$：放不下，$dp[2][0] = 0$
- $j = 1$：$dp[2][1] = \max(dp[1][1], dp[1][0] + 2) = \max(0, 2) = 2$
- $j = 2$：$dp[2][2] = \max(dp[1][2], dp[1][1] + 2) = \max(3, 2) = 3$
- $j = 3$：$dp[2][3] = \max(dp[1][3], dp[1][2] + 2) = \max(3, 5) = 5$
- $j = 4$：$dp[2][4] = \max(dp[1][4], dp[1][3] + 2) = \max(3, 5) = 5$
- $j = 5$：$dp[2][5] = \max(dp[1][5], dp[1][4] + 2) = \max(3, 5) = 5$

$i \backslash j$	0	1	2	3	4	5	
0	0	0	0	0	0	0	（无物品）
1	0	0	3	3	3	3	$w_1 = 2, v_1 = 3$
2	0	2	3	5	5	5	$w_2 = 1, v_2 = 2$
3	0	2	3	5	6	7	$w_3 = 3, v_3 = 4$
4	0	2	3	5	6	$\boxed{8}$	答案 $w_4 = 2, v_4 = 3$

选物品 2

不选

图 10：0-1 背包 DP 表格：$dp[4][5] = 8$，选物品 1，2，4（重量 $2 + 1 + 2 = 5$，价值 $3 + 2 + 3 = 8$）

回溯：找出选了哪些物品

填完表后，从 $dp[n][W]$ 往回追溯：

```
j ← W
for i = n down to 1 do
    if dp[i][j] ≠ dp[i − 1][j] then
        输出" 选择物品 i"
        j ← j − w[i]
    end if
end for
```

例子中的回溯：

- $dp[4][5] = 8 \neq dp[3][5] = 7 \rightarrow$ 选物品 4，$j \leftarrow 5 - 2 = 3$

- $dp[3][3] = 5 = dp[2][3] = 5 \to$ 不选物品 3
- $dp[2][3] = 5 \neq dp[1][3] = 3 \to$ 选物品 2，$j \leftarrow 3 - 1 = 2$
- $dp[1][2] = 3 \neq dp[0][2] = 0 \to$ 选物品 1，$j \leftarrow 2 - 2 = 0$

结果：选物品 1, 2, 4，总重量 5，总价值 8。

0-1 背包的最优子结构证明

命题：设 S^* 是最优解（物品子集），总重量 $\leq W$，总价值最大。
若物品 $i \in S^*$，则 $S^* \setminus \{i\}$ 是前 $i-1$ 个物品、容量 $W - w_i$ 的最优解。
证明（剪切-粘贴）：
假设 $S^* \setminus \{i\}$ 不是容量 $W - w_i$ 的最优解。
则存在 S'，只用前 $i-1$ 个物品，总重量 $\leq W - w_i$，且 $v(S') > v(S^* \setminus \{i\})$。
构造 $S'' = S' \cup \{i\}$：

- 总重量 $\leq W - w_i + w_i = W$
- 总价值 $= v(S') + v_i > v(S^* \setminus \{i\}) + v_i = v(S^*)$

这与 S^* 是最优解矛盾！□

核心结论：DP 证明的通用框架

Step 1：最优子结构

- 用剪切-粘贴法证明：最优解包含子问题的最优解

Step 2：状态转移方程

- 枚举”最后一步”的所有可能
- 每种可能对应一个子问题
- 取最优（最大/最小）

Step 3：归纳法验证

- 基础情况：边界条件正确
- 归纳步骤：假设子问题正确，证明当前问题正确

Step 4：复杂度分析

- 状态数量 × 每个状态的转移代价

7.5.1 0-1 背包的 FPTAS

动机：伪多项式 DP 能否变成真正的多项式近似？

0-1 背包的精确 DP 是 $O(nW)$，是**伪多项式时间**——当 W 指数大时不可行。
能否在多项式时间内找到**接近最优**的解？
答案：可以！通过**值域缩放**得到一个 FPTAS：给定 $\varepsilon > 0$，在 $O(n^2/\varepsilon)$ 时间内找到价值 $\geq (1-\varepsilon)\,\mathrm{OPT}$ 的解。

FPTAS 算法：值域缩放

输入：物品重量 $w[1..n]$，价值 $v[1..n]$，背包容量 W，误差参数 $\varepsilon > 0$
输出：价值 $\geq (1-\varepsilon)\,\mathrm{OPT}$ 的可行解
Step 1：令 $V_{\max} = \max_i v_i$，缩放因子 $K = \dfrac{\varepsilon \cdot V_{\max}}{n}$
Step 2：对每个物品取整缩放值

$$v_i' = \left\lfloor \frac{v_i}{K} \right\rfloor$$

Step 3：对缩放后的价值跑**最小重量 DP**：

$$dp[v] = \text{用若干物品恰好达到总缩放价值} v \text{ 的最小重量}$$

```
dp[0] ← 0;  dp[v] ← ∞  (v > 0)
for i = 1 to n do
    for v from V'_max downto v'_i do
        dp[v] ← min(dp[v], dp[v − v'_i] + w_i)
    end for
end for
return max{v : dp[v] ≤ W} 对应的原始价值
```

其中 $V'_{\max} = \sum_i v_i' \leq n \cdot \dfrac{n}{\varepsilon} = \dfrac{n^2}{\varepsilon}$

近似比证明

设 S^* 为最优解，$\mathrm{OPT} = \sum_{i \in S^*} v_i$。
关键观察：每个物品取整损失 $< K$，最优解 $|S^*| \leq n$，故总损失

$$\mathrm{OPT} - K \cdot \sum_{i \in S^*} v_i' < n \cdot K = \varepsilon \cdot V_{\max} \leq \varepsilon \cdot \mathrm{OPT}$$

（最后一步：$V_{\max} \leq \mathrm{OPT}$，因为最优解至少包含一个物品）

设算法解为 S'，在缩放问题中最优，故

$$\text{ALG} \;\geq\; K \cdot \sum_{i \in S'} v'_i \;\geq\; K \cdot \sum_{i \in S^*} v'_i \;\geq\; \text{OPT} - \varepsilon\,\text{OPT} = (1-\varepsilon)\,\text{OPT} \quad \square$$

复杂度：0-1 背包 FPTAS

- 每个缩放价值 $v'_i \leq V_{\max}/K = n/\varepsilon$
- 缩放后总价值上界：$n \cdot (n/\varepsilon) = n^2/\varepsilon$
- DP 表大小：$O(n^2/\varepsilon)$，每格 $O(1)$
- **总时间复杂度**：$O(n^2/\varepsilon)$——关于 n 和 $1/\varepsilon$ 均为多项式

核心结论：为什么 0-1 背包能有 FPTAS？

- 0-1 背包是**弱 NP 难**（NP 难性依赖于输入数值的大小，而非规模）
- 弱 NP 难问题可以通过”缩放 + 取整”把大数值问题转化为小数值问题
- **强 NP 难**问题（如一般 TSP、图着色）除非 P = NP，否则不存在 FPTAS

问题	NP 难类型	有无 FPTAS
0-1 背包	弱 NP 难	有（本节算法）
分数背包	多项式可解	不需要
度量 TSP	强 NP 难	无（有 3/2-近似）
一般 TSP	强 NP 难	无近似（除非 P=NP）

7.6 更多 DP 经典例子

7.6.1 例：爬楼梯（最基础线性 DP）

爬楼梯问题

问题：楼梯有 n 阶，每次可以爬 1 阶或 2 阶。有多少种不同的方法爬到顶？
状态定义：$dp[i]$ = 爬到第 i 阶的方法数
状态转移：

$$dp[i] = dp[i-1] + dp[i-2]$$

解释：到达第 i 阶，要么从第 $i-1$ 阶爬 1 步，要么从第 $i-2$ 阶爬 2 步。
边界：$dp[1] = 1$，$dp[2] = 2$
本质：这就是 Fibonacci 数列！

爬楼梯手算

输入：$n = 5$

i	1	2	3	4	5
$dp[i]$	1	2	3	5	8

计算：

- $dp[3] = dp[2] + dp[1] = 2 + 1 = 3$
- $dp[4] = dp[3] + dp[2] = 3 + 2 = 5$
- $dp[5] = dp[4] + dp[3] = 5 + 3 = 8$

答案：8 种方法
空间优化：只需保存前两个值，$O(1)$ 空间

7.6.2 例：最大子数组和

最大子数组和 (Kadane 算法)

问题：给定数组 $A[1..n]$（可能有负数），求**连续**子数组的最大和。
例：$A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$
最大子数组：$[4, -1, 2, 1]$，和 $= 6$
状态定义：$dp[i]$ = 以 $A[i]$ **结尾**的最大子数组和
状态转移：

$$dp[i] = \max(dp[i-1] + A[i], A[i])$$

解释：要么延续前面的子数组，要么从 $A[i]$ 重新开始。
最终答案：$\max_i dp[i]$

Kadane 算法手算

输入：$A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

i	1	2	3	4	5	6	7	8	9
$A[i]$	-2	1	-3	4	-1	2	1	-5	4
$dp[i]$	-2	1	-2	4	3	5	6	1	5

逐步：

- $dp[1] = -2$（只能选-2）
- $dp[2] = \max(-2 + 1, 1) = \max(-1, 1) = 1$（重新开始更好）
- $dp[3] = \max(1 - 3, -3) = \max(-2, -3) = -2$（延续更好）
- $dp[4] = \max(-2 + 4, 4) = \max(2, 4) = 4$（重新开始）
- $dp[5] = \max(4 - 1, -1) = 3$
- $dp[6] = \max(3 + 2, 2) = 5$
- $dp[7] = \max(5 + 1, 1) = 6 \leftarrow$ **最大值**

答案：6，对应子数组 $[4, -1, 2, 1]$
时间复杂度：$O(n)$，空间 $O(1)$（只需记录 $dp[i-1]$）

7.6.3 例：打家劫舍（线性 DP 带约束）

打家劫舍问题（LeetCode 198）

问题：n 间房子排成一排，第 i 间有 $nums[i]$ 元。相邻两间不能同时偷。求最大金额。

例：$nums = [2, 7, 9, 3, 1]$

枚举几种方案：

- 偷第 1、3、5 间：$2 + 9 + 1 = 12$
- 偷第 1、3 间：$2 + 9 = 11$
- 偷第 2、4 间：$7 + 3 = 10$
- 偷第 2、5 间：$7 + 1 = 8$

方案数随 n 指数增长，不能暴力枚举——需要 DP。

状态定义：$dp[i]$ = 前 i 间房子能偷的最大金额

打家劫舍状态转移

状态转移：

$$dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$$

解释：

- $dp[i-1]$：不偷第 i 间
- $dp[i-2] + nums[i]$：偷第 i 间（则第 $i-1$ 间不能偷）

边界：$dp[0] = nums[0]$，$dp[1] = \max(nums[0], nums[1])$

打家劫舍手算

输入：$nums = [2, 7, 9, 3, 1]$

i	0	1	2	3	4
$nums[i]$	2	7	9	3	1
$dp[i]$	2	7	11	11	12

计算：

- $dp[0] = 2$
- $dp[1] = \max(2, 7) = 7$
- $dp[2] = \max(dp[1], dp[0] + 9) = \max(7, 11) = 11$（偷 0 和 2）

- $dp[3] = \max(dp[2], dp[1] + 3) = \max(11, 10) = 11$（不偷 3）
- $dp[4] = \max(dp[3], dp[2] + 1) = \max(11, 12) = 12$（偷 2 和 4）

答案：12（偷第 0、2、4 间：$2 + 9 + 1 = 12$）

7.6.4 例：Nim 取石子（博弈 DP）

Nim 取石子问题

问题：有 n 颗石子，两名玩家轮流取，每次可取 1、2 或 3 颗。取走最后一颗石子的玩家**获胜**。问先手是否必胜？
状态定义：$dp[i]$ = 当前还剩 i 颗石子时，**当前玩家**是否必胜
边界：$dp[0] = \text{False}$（无石子可取，当前玩家输）
状态转移：当前玩家可取 $1, 2, 3$ 颗，只要存在**一种取法**让对手进入必败态，当前玩家就必胜：

$$dp[i] = \neg dp[i-1] \ \vee \ \neg dp[i-2] \ \vee \ \neg dp[i-3]$$

等价地：$dp[i] = \text{True}$ 当且仅当 $dp[i-1], dp[i-2], dp[i-3]$ 中**至少有一个为 False**。

Nim 手算前 8 项

i	0	1	2	3	4	5	6	7	8
$dp[i]$	F	W	W	W	F	W	W	W	F

逐步分析：

- $dp[0] = \text{F}$（无子可取，必败）
- $dp[1] = \text{W}$（取 1 颗，对手面对 $dp[0] = \text{F}$）
- $dp[2] = \text{W}$（取 2 颗，对手面对 $dp[0] = \text{F}$）
- $dp[3] = \text{W}$（取 3 颗，对手面对 $dp[0] = \text{F}$）
- $dp[4] = \text{F}$（取 1/2/3 颗后对手分别面对 $dp[3], dp[2], dp[1]$，全为 W）
- 规律：每 4 个一循环，$dp[i] = \text{F}$ 当且仅当 $4 \mid i$

结论：先手必败 $\iff n \equiv 0 \pmod 4$

Nim 的最优子结构证明

命题：$dp[i]$ 正确描述了必胜/必败态。

证明（归纳）：

情形 1（必败）：若 $dp[i-1] = dp[i-2] = dp[i-3] = \mathrm{W}$，则无论取几颗，对手均处于必胜态，当前玩家必败，$dp[i] = \mathrm{F}$。

情形 2（必胜）：若存在 $k \in \{1, 2, 3\}$ 使 $dp[i-k] = \mathrm{F}$，则取 k 颗后对手进入必败态，$dp[i] = \mathrm{W}$。

两情形覆盖所有情况，归纳即得全部 $dp[i]$ 正确。□

复杂度：Nim 取石子

- 时间：$O(n)$（逐项填表）
- 空间：$O(1)$（只需前 3 项）
- **闭合公式**：$dp[i] = (i \bmod 4 \neq 0)$

7.6.5 例：硬币找零

硬币找零问题

问题：给定硬币面值 $coins = [c_1, c_2, \ldots, c_k]$（每种无限个），求凑成金额 $amount$ 的最少硬币数。

例：$coins = [1, 2, 5]$，$amount = 11$

最优解：$11 = 5 + 5 + 1$，需要 3 枚硬币

状态定义：$dp[i]$ = 凑成金额 i 的最少硬币数

状态转移：

$$dp[i] = \min_{c \in coins, c \leq i} (dp[i-c] + 1)$$

边界：$dp[0] = 0$

硬币找零手算

输入：$coins = [1, 2, 5]$，$amount = 11$

i	0	1	2	3	4	5	6	7	8	9	10	11
$dp[i]$	0	1	1	2	2	1	2	2	3	3	2	3

关键步骤：

- $dp[1] = dp[1-1] + 1 = dp[0] + 1 = 1$（用 1 枚 1 元）
- $dp[2] = \min(dp[2-1] + 1, dp[2-2] + 1) = \min(2, 1) = 1$（用 1 枚 2 元）

- $dp[5] = \min(dp[4]+1, dp[3]+1, dp[0]+1) = \min(3,3,1) = 1$（用 1 枚 5 元）
- $dp[11] = \min(dp[10]+1, dp[9]+1, dp[6]+1) = \min(3,4,3) = 3$

答案：3 枚硬币（5+5+1）

硬币找零的最优子结构证明

命题：设凑成金额 n 的最优解使用了硬币 c，则去掉这枚硬币后，剩余部分是凑成 $n-c$ 的最优解。
证明（剪切-粘贴）：
设最优解用了 k 枚硬币凑成 n，其中一枚是 c。
假设剩余的 $k-1$ 枚不是凑成 $n-c$ 的最优解。
则存在用 $k' < k-1$ 枚硬币凑成 $n-c$ 的方案。
那么 $k'+1 < k$ 枚硬币就能凑成 n，与 k 是最优矛盾。□

7.6.6 例：最长递增子序列 (LIS)

LIS 问题定义

问题：给定数组 $A[1..n]$，求最长**严格递增**子序列的长度。
子序列：可以不连续，但必须保持原顺序。
例：$A = [10, 9, 2, 5, 3, 7, 101, 18]$
LIS $= [2, 3, 7, 18]$ 或 $[2, 3, 7, 101]$ 或 $[2, 5, 7, 18]$，长度 $= 4$

LIS 的 DP 解法

状态定义：$dp[i]$ = 以 $A[i]$ **结尾**的 LIS 长度
状态转移：

$$dp[i] = \max_{j<i, A[j]<A[i]} (dp[j]+1)$$

如果不存在这样的 j，则 $dp[i] = 1$（只有 $A[i]$ 自己）
最终答案：$\max_i dp[i]$
时间复杂度：$O(n^2)$（可用二分优化到 $O(n \log n)$）

LIS 手算例子

输入：$A = [10, 9, 2, 5, 3, 7, 101, 18]$

i	1	2	3	4	5	6	7	8
$A[i]$	10	9	2	5	3	7	101	18
$dp[i]$	1	1	1	2	2	3	4	4

逐步计算：

- $dp[1] = 1$（只有 10 自己）
- $dp[2] = 1$（9 前面没有比它小的）
- $dp[3] = 1$（2 前面没有比它小的）
- $dp[4] = \max(dp[3]) + 1 = 2$（5 前面有 $2 < 5$）
- $dp[5] = \max(dp[3]) + 1 = 2$（3 前面有 $2 < 3$）
- $dp[6] = \max(dp[3], dp[4], dp[5]) + 1 = 3$（7 前面有 2,5,3 都 < 7，取最大的 dp 值 +1）
- $dp[7] = \max(dp[1], \ldots, dp[6]) + 1 = 4$（101 比所有都大）
- $dp[8] = \max(dp[3], dp[4], dp[5], dp[6]) + 1 = 4$（$18 >$ 2,5,3,7）

答案：$\max(dp) = 4$

7.6.7 例：矩阵链乘法

矩阵链乘法问题

问题：n 个矩阵 $A_1, A_2, \ldots, A_n$，其中 A_i 的维度是 $p_{i-1} \times p_i$。
求计算乘积 $A_1 A_2 \cdots A_n$ 的最少标量乘法次数。
关键：矩阵乘法满足结合律，不同加括号方式代价不同！
例：三个矩阵，维度 10×30, 30×5, 5×60

- $(A_1 A_2) A_3$：$10 \times 30 \times 5 + 10 \times 5 \times 60 = 1500 + 3000 = 4500$
- $A_1 (A_2 A_3)$：$30 \times 5 \times 60 + 10 \times 30 \times 60 = 9000 + 18000 = 27000$

第一种方式快 6 倍！

矩阵链乘法的 DP

状态定义：$dp[i][j] =$ 计算 $A_i A_{i+1} \cdots A_j$ 的最少乘法次数
状态转移：枚举最后一次乘法的分割点 k

$$dp[i][j] = \min_{i \le k < j} \left(dp[i][k] + dp[k+1][j] + p_{i-1} \cdot p_k \cdot p_j \right)$$

解释：

- $dp[i][k]$：计算左半部分 $A_i \cdots A_k$ 的代价
- $dp[k+1][j]$：计算右半部分 $A_{k+1} \cdots A_j$ 的代价
- $p_{i-1} \cdot p_k \cdot p_j$：合并两个结果矩阵的代价

边界：$dp[i][i] = 0$（单个矩阵无需乘法）
计算顺序：按区间长度从小到大

矩阵链乘法手算例子

输入：4 个矩阵，维度数组 $p = [30, 35, 15, 5, 10]$
即 $A_1 : 30 \times 35$，$A_2 : 35 \times 15$，$A_3 : 15 \times 5$，$A_4 : 5 \times 10$
长度 1：$dp[1][1] = dp[2][2] = dp[3][3] = dp[4][4] = 0$
长度 2：

- $dp[1][2] = p_0 \cdot p_1 \cdot p_2 = 30 \times 35 \times 15 = 15750$
- $dp[2][3] = p_1 \cdot p_2 \cdot p_3 = 35 \times 15 \times 5 = 2625$
- $dp[3][4] = p_2 \cdot p_3 \cdot p_4 = 15 \times 5 \times 10 = 750$

长度 3：

- $dp[1][3] = \min \begin{cases} dp[1][1] + dp[2][3] + 30 \times 35 \times 5 = 0 + 2625 + 5250 = 7875 \\ dp[1][2] + dp[3][3] + 30 \times 15 \times 5 = 15750 + 0 + 2250 = 18000 \end{cases} = 7875$
- $dp[2][4] = \min \begin{cases} 0 + 750 + 35 \times 15 \times 10 = 6000 \\ 2625 + 0 + 35 \times 5 \times 10 = 4375 \end{cases} = 4375$

长度 4：

$$dp[1][4] = \min \begin{cases} dp[1][1] + dp[2][4] + 30 \times 35 \times 10 = 0 + 4375 + 10500 = 14875 \\ dp[1][2] + dp[3][4] + 30 \times 15 \times 10 = 15750 + 750 + 4500 = 21000 \\ dp[1][3] + dp[4][4] + 30 \times 5 \times 10 = 7875 + 0 + 1500 = 9375 \end{cases} = 9375$$

答案：9375 次乘法，最优分割：$((A_1(A_2A_3))A_4)$

7.7 DP 的六种类型总结

核心结论：DP 类型分类

1. **线性 DP**：状态按一维顺序推进
   - 形式：$dp[i] = f(dp[i-1], dp[i-2], \ldots)$
   - 例：Fibonacci、爬楼梯、打家劫舍、Kadane、LIS
2. **二维 DP**：状态有两个自由度
   - 形式：$dp[i][j]$
   - 例：0-1 背包、编辑距离、LCS、网格路径
3. **区间 DP**：状态是一个区间 $[l, r]$
   - 形式：$dp[l][r] = \min_k \{dp[l][k] + dp[k+1][r] + cost\}$
   - 例：矩阵链乘法、石子合并、括号匹配
4. **状态压缩 DP**：状态用二进制集合表示
   - 形式：$dp[mask]$ 或 $dp[mask][i]$
   - 例：TSP、子集覆盖、博弈
5. **树形 DP**：状态定义在树节点
   - 形式：$dp[u] = \text{combine}(dp[v_1], dp[v_2], \ldots)$
   - 例：树上最大独立集、树上背包
6. **DAG DP**：状态在有向无环图
   - 形式：$dp[v] = \min_{u \to v} dp[u] + w(u, v)$
   - 例：DAG 最长路径、拓扑序 DP

7.8 状态压缩 DP：旅行商问题

旅行商问题 (TSP)

问题：给定 n 个城市和城市间的距离矩阵 $d[i][j]$。
从城市 0 出发，访问每个城市**恰好一次**，最后回到城市 0。求最短路径长度。
暴力：枚举所有 $(n-1)!$ 种排列，$O(n!)$
状态压缩 DP：$O(n^2 \cdot 2^n)$，对 $n \leq 20$ 可行
核心思想：用**二进制掩码**表示已访问城市的集合

TSP 状态定义

状态：$dp[S][i]$ = 从城市 0 出发，访问了集合 S 中的城市，当前在城市 i 的最短距离
其中 S 是二进制数，第 j 位为 1 表示城市 j 已访问
状态转移：

$$dp[S][i] = \min_{j \in S, j \neq i} (dp[S \setminus \{i\}][j] + d[j][i])$$

解释：到达 (S, i) 的最短路 = 从某个 $j \in S$ 走过来
边界：$dp[\{0\}][0] = 0$（从城市 0 出发，只访问了城市 0）
答案：$\min_i (dp[\{0, 1, \ldots, n-1\}][i] + d[i][0])$（回到城市 0）

TSP 手算例子

输入：4 个城市，距离矩阵

d	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

集合用二进制表示：$S = \{0, 2\} \to 0101_2 = 5$
初始化：$dp[0001][0] = 0$（在城市 0，只访问了 $\{0\}$）
扩展到 2 个城市：

- $dp[0011][1] = dp[0001][0] + d[0][1] = 0 + 10 = 10$
- $dp[0101][2] = dp[0001][0] + d[0][2] = 0 + 15 = 15$
- $dp[1001][3] = dp[0001][0] + d[0][3] = 0 + 20 = 20$

扩展到 3 个城市（完整计算所有包含城市 0 的 3 元子集）：

- $S = \{0,1,2\} = 0111$：
  - $dp[0111][1] = dp[0101][2] + d[2][1] = 15 + 35 = 50$（由城市 2 出发到城市 1）
  - $dp[0111][2] = dp[0011][1] + d[1][2] = 10 + 35 = 45$（由城市 1 出发到城市 2）
- $S = \{0,1,3\} = 1011$：
  - $dp[1011][1] = dp[1001][3] + d[3][1] = 20 + 25 = 45$（由城市 3 出发到城市 1）
  - $dp[1011][3] = dp[0011][1] + d[1][3] = 10 + 25 = 35$（由城市 1 出发到城市 3）
- $S = \{0,2,3\} = 1101$：
  - $dp[1101][2] = dp[1001][3] + d[3][2] = 20 + 30 = 50$（由城市 3 出发到城市 2）
  - $dp[1101][3] = dp[0101][2] + d[2][3] = 15 + 30 = 45$（由城市 2 出发到城市 3）

TSP 完整计算

扩展到 4 个城市（全集 $S = 1111_2 = 15$，即 $S = \{0,1,2,3\}$）：
对每个终点城市 i，枚举所有到达 i 前的上一城市 $j \in S \setminus \{i\}, j \neq 0$（城市 0 只能是起点）：

- $dp[1111][1] = \min \begin{cases} dp[1101][2] + d[2][1] = 50 + 35 = 85 \\ dp[1101][3] + d[3][1] = 45 + 25 = 70 \end{cases} = \mathbf{70}$
- $dp[1111][2] = \min \begin{cases} dp[1011][1] + d[1][2] = 45 + 35 = 80 \\ dp[1011][3] + d[3][2] = 35 + 30 = 65 \end{cases} = \mathbf{65}$
- $dp[1111][3] = \min \begin{cases} dp[0111][1] + d[1][3] = 50 + 25 = 75 \\ dp[0111][2] + d[2][3] = 45 + 30 = 75 \end{cases} = \mathbf{75}$

答案：加上最后回到城市 0 的费用

$$\min(dp[1111][1] + d[1][0],\ dp[1111][2] + d[2][0],\ dp[1111][3] + d[3][0])$$

$$= \min(70 + 10,\ 65 + 15,\ 75 + 20) = \min(80, 80, 95) = \mathbf{80}$$

最优路径（两条等价）：

- $0 \to 1 \to 3 \to 2 \to 0$：$d[0][1]+d[1][3]+d[3][2]+d[2][0] = 10+25+30+15 = 80$
- $0 \to 2 \to 3 \to 1 \to 0$：$d[0][2]+d[2][3]+d[3][1]+d[1][0] = 15+30+25+10 = 80$

TSP 复杂度分析

状态数：$O(2^n \cdot n)$

- 2^n 种子集
- 每种子集有 n 种" 当前城市"

转移：每个状态需要 $O(n)$ 时间枚举上一个城市
总复杂度：$O(n^2 \cdot 2^n)$
对比暴力：

n	暴力 $(n-1)!$	状压 DP $n^2 2^n$
10	362,880	102,400
15	8.7×10^{10}	7.4×10^6
20	1.2×10^{17}	4.2×10^8

$n = 20$ 时，状压 DP 快了 10^8 倍！

7.9 树形 DP：最大独立集

树上最大独立集问题

问题：给定一棵 n 个节点的树，每个节点有权值 $w[i]$。
选择一些节点，使得任意两个被选节点**不相邻**，且权值和最大。
类比：打家劫舍的树形版本！
例：公司聚会问题——员工形成上下级树，上下级不能同时参加

树形 DP 状态定义

状态：对每个节点 u 定义两个值

- $dp[u][0]$ = 不选 u 时，u 的子树的最大权值和
- $dp[u][1]$ = 选 u 时，u 的子树的最大权值和

状态转移：设 $v_1, v_2, \ldots$ 是 u 的子节点

$$dp[u][0] = \sum_v \max(dp[v][0], dp[v][1])$$

$$dp[u][1] = w[u] + \sum_v dp[v][0]$$

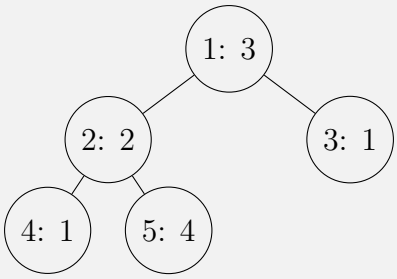
解释：

- 不选 u：每个子节点可选可不选
- 选 u：所有子节点都不能选

答案：$\max(dp[root][0], dp[root][1])$

树形 DP 手算例子

输入：5 个节点的树，权值如图



后序遍历计算（先算叶子，再算父节点）：
叶子节点：

- $dp[4][0] = 0$，$dp[4][1] = 1$
- $dp[5][0] = 0$，$dp[5][1] = 4$
- $dp[3][0] = 0$，$dp[3][1] = 1$

节点 2（子节点：4, 5）：

- $dp[2][0] = \max(0, 1) + \max(0, 4) = 1 + 4 = 5$
- $dp[2][1] = 2 + 0 + 0 = 2$

节点 1（根）（子节点：2, 3）：

- $dp[1][0] = \max(5, 2) + \max(0, 1) = 5 + 1 = 6$
- $dp[1][1] = 3 + 5 + 0 = 8$

答案：$\max(dp[1][0], dp[1][1]) = \max(6, 8) = 8$
回溯最优选择：答案来自 $dp[1][1] = 8$，即**选节点 1**（权值 3）。

- 选了节点 1 ⇒ 其子节点 2 和 3 **不能选**（与节点 1 相邻）。
- 对节点 2 的子树取 $dp[2][0] = 5$（不选节点 2 时的最优值）：

  $$dp[2][0] = \max(dp[4][0], dp[4][1]) + \max(dp[5][0], dp[5][1]) = \max(0, 1) + \max(0, 4) = 1 + 4 = 5$$

  即节点 4（权值 1）和节点 5（权值 4）各自独立，**两者均选**。
- 对节点 3 的子树取 $dp[3][0] = 0$（不选节点 3，且节点 3 为叶子，贡献为 0）。

因此 $dp[1][1] = w[1] + dp[2][0] + dp[3][0] = 3 + 5 + 0 = 8$。
最终答案：选节点集合 $\{1, 4, 5\}$，权值之和 $= 3 + 1 + 4 = 8$。
合法性验证：节点 1 与 4 不相邻（树中路径为 1-2-4，无直接边），节点 1 与 5 不相邻（路径为 1-2-5），节点 4 与 5 不相邻（树中无边 4-5）。✓

7.10 DAG DP：最长路径

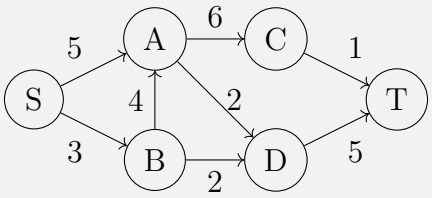
DAG 最长路径问题

问题：给定有向无环图 (DAG)，边有权重。求从源点到汇点的最长路径。
注意：一般图的最长路径是 NP-hard，但 DAG 上可以用 DP 在 $O(V + E)$ 解决！
核心：DAG 可以拓扑排序，按拓扑序 DP
状态：$dp[v]$ = 从源点到 v 的最长路径长度
转移：

$$dp[v] = \max_{u \to v}(dp[u] + w(u, v))$$

DAG 最长路径例子

输入：6 个节点的 DAG



拓扑排序：S → B → A → D → C → T

DAG 最长路径手算

按拓扑序计算：

节点	dp 值计算
S	$dp[S] = 0$（源点）
B	$dp[B] = dp[S] + 3 = 3$
A	$dp[A] = \max(dp[S] + 5, dp[B] + 4) = \max(5, 7) = 7$
D	$dp[D] = \max(dp[A] + 2, dp[B] + 2) = \max(9, 5) = 9$
C	$dp[C] = dp[A] + 6 = 13$
T	$dp[T] = \max(dp[C] + 1, dp[D] + 5) = \max(14, 14) = 14$

答案：最长路径长度 $= 14$
最长路径：$\mathrm{S} \to \mathrm{B} \to \mathrm{A} \to \mathrm{C} \to \mathrm{T}$（$3 + 4 + 6 + 1 = 14$）
或 $\mathrm{S} \to \mathrm{B} \to \mathrm{A} \to \mathrm{D} \to \mathrm{T}$（$3 + 4 + 2 + 5 = 14$）

DAG DP 的应用

1. 项目调度（关键路径法 CPM）

- 节点 = 任务，边 = 依赖关系
- 最长路径 = 项目最短完成时间
- 关键路径上的任务不能延迟

2. 最长递增子序列（LIS）

- 构造 DAG：若 $i < j$ 且 $A[i] < A[j]$，则 $i \to j$
- LIS 长度 = DAG 最长路径 + 1

3. 课程规划

- 节点 = 课程，边 = 先修关系
- 最长路径 = 最少需要几个学期

时间复杂度：$O(V + E)$（拓扑排序 + 一次遍历）

8 DP 的 GPU 困境与突破

8.1 为什么 DP 难以并行？

DP 的依赖结构

问题：$dp[i][j]$ 依赖于 $dp[i-1][j]$, $dp[i][j-1]$, $dp[i-1][j-1]$
这意味着：

- 不能同时计算 $dp[i][j]$ 和 $dp[i-1][j]$（有依赖）
- 不能同时计算 $dp[i][j]$ 和 $dp[i][j-1]$（有依赖）

传统方法：逐行或逐列计算，$O(mn)$ 步，无并行！
关键洞察：虽然不能并行计算整行/整列，但可以并行计算**对角线**！

8.2 对角线并行（Wavefront 并行）

Wavefront 并行的原理

观察：同一条**反对角线**上的元素**互不依赖**！
定义第 k 条对角线：$\{(i,j) : i+j=k\}$
对于 (i,j) 满足 $i+j=k$：

- 依赖 $(i-1,j)$：$i-1+j=k-1$ 前一条对角线
- 依赖 $(i,j-1)$：$i+j-1=k-1$ 前一条对角线
- 依赖 $(i-1,j-1)$：$i-1+j-1=k-2$ 更早的对角线

结论：第 k 条对角线只依赖第 $k-1$ 和 $k-2$ 条，不依赖同一条！
并行算法：

1. 按对角线顺序 $k=0,1,2,\ldots,m+n-2$
2. 每条对角线内，所有元素**并行计算**

复杂度：Wavefront 并行的复杂度

总对角线数：$m+n-1$
最长对角线：$\min(m,n)$ 个元素
并行时间：$O(m+n)$（假设足够多处理器）
加速比：$\frac{O(mn)}{O(m+n)} = O(\min(m,n))$

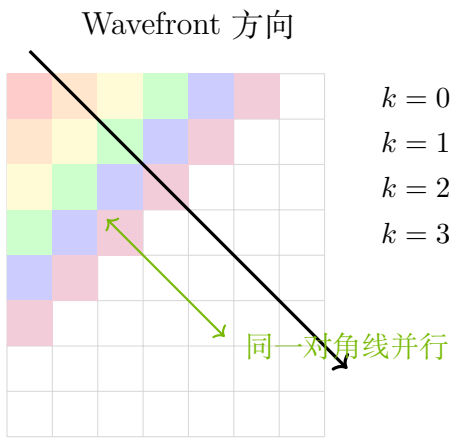


图 11: Wavefront 并行：沿对角线方向推进，每条对角线内部并行

例：$m = n = 10000$

- 串行：10^8 步
- Wavefront 并行：2×10^4 步
- 加速比：5000×

实验：编辑距离 GPU 化

```python
import cupy as cp
import numpy as np
import time

def edit_distance_cpu(A, B):
    """CPU版本：标准DP"""
    m, n = len(A), len(B)
    dp = np.zeros((m+1, n+1), dtype=np.int32)
    dp[0, :] = np.arange(n+1)
    dp[:, 0] = np.arange(m+1)

    for i in range(1, m+1):
        for j in range(1, n+1):
            if A[i-1] == B[j-1]:
                dp[i, j] = dp[i-1, j-1]
            else:
                dp[i, j] = 1 + min(dp[i-1, j], dp[i, j-1], dp[i
                    -1, j-1])
```

```python
    return dp[m, n]

def edit_distance_gpu_wavefront(A, B):
    """GPU版本：对角线并行"""
    m, n = len(A), len(B)

    # 转换为数值数组
    A_arr = cp.array([ord(c) for c in A], dtype=cp.int32)
    B_arr = cp.array([ord(c) for c in B], dtype=cp.int32)

    # DP表格
    dp = cp.zeros((m+1, n+1), dtype=cp.int32)
    dp[0, :] = cp.arange(n+1)
    dp[:, 0] = cp.arange(m+1)

    # 对角线并行
    for k in range(2, m + n + 1):
        # 当前对角线的有效范围
        i_start = max(1, k - n)
        i_end = min(m, k - 1)

        if i_start > i_end:
            continue

        # 构造索引
        i_indices = cp.arange(i_start, i_end + 1)
        j_indices = k - i_indices

        # 并行计算这条对角线上的所有元素
        match = (A_arr[i_indices - 1] == B_arr[j_indices - 1]).
            astype(cp.int32)

        candidates = cp.stack([
            dp[i_indices - 1, j_indices] + 1,       # 删除
            dp[i_indices, j_indices - 1] + 1,       # 插入
            dp[i_indices - 1, j_indices - 1] + (1 - match)  # 匹
                配/替换
        ])
```

```python

        dp[i_indices, j_indices] = cp.min(candidates, axis=0)

    return int(dp[m, n])

# 测试
m, n = 5000, 5000
A = ''.join(np.random.choice(list('ACGT'), m))
B = ''.join(np.random.choice(list('ACGT'), n))

# CPU
start = time.time()
dist_cpu = edit_distance_cpu(A, B)
time_cpu = time.time() - start
print(f"CPU: {dist_cpu}, time: {time_cpu:.2f}s")

# GPU
cp.cuda.Stream.null.synchronize()
start = time.time()
dist_gpu = edit_distance_gpu_wavefront(A, B)
cp.cuda.Stream.null.synchronize()
time_gpu = time.time() - start
print(f"GPU: {dist_gpu}, time: {time_gpu:.2f}s")
print(f"Speedup: {time_cpu/time_gpu:.1f}x")
```

典型结果 $(m = n = 5000)$：

- CPU: ∼15s
- GPU (Wavefront): ∼0.3s
- 加速比：$\sim 50\times$

9 贪心算法

9.1 贪心的核心思想

贪心算法的结构

贪心策略：在每一步选择**当前看起来最优**的选项。
与 DP 的区别：

- DP：考虑所有子问题，选最优组合
- 贪心：只做一次选择，不回溯

贪心正确的条件：

1. **贪心选择性质**：局部最优选择能导向全局最优
2. **最优子结构**：问题的最优解包含子问题的最优解

证明技巧：交换论证
假设存在一个不用贪心选择的最优解，证明可以”交换”成用贪心选择的解，且不变差。

9.2 例：活动选择问题

问题定义

问题：有 n 个活动，第 i 个活动的时间段是 $[s_i, f_i)$（从 s_i 开始，f_i 结束）。
同一时刻只能参加一个活动。求最多能参加多少个活动。
冲突定义：活动 i 和 j 冲突 $\Leftrightarrow [s_i, f_i) \cap [s_j, f_j) \neq \emptyset$
目标：选择最大的**互不冲突**的活动子集。

具体例子

输入：11 个活动

活动 i	1	2	3	4	5	6	7	8	9	10	11
开始 s_i	1	3	0	5	3	5	6	8	8	2	12
结束 f_i	4	5	6	7	9	9	10	11	12	14	16

按结束时间排序后：

活动 i	1	2	3	4	5	6	7	8	9	10	11
开始 s_i	1	3	0	5	3	5	6	8	8	2	12
结束 f_i	4	5	6	7	9	9	10	11	12	14	16

（这个例子恰好已经按 f_i 排好序了）

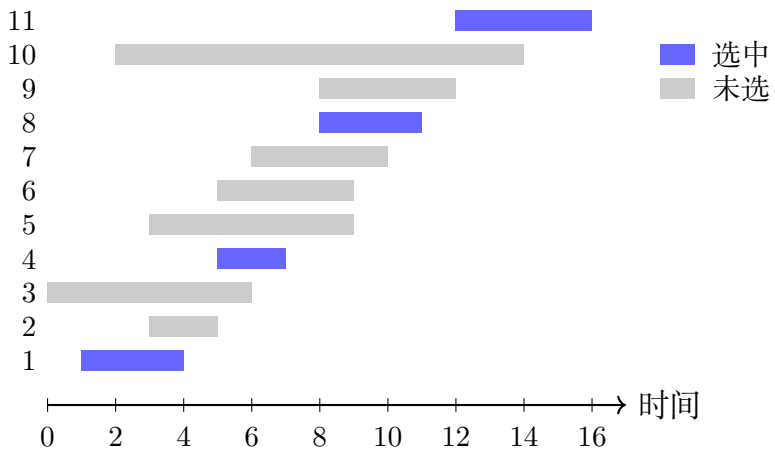


图 12: 活动选择：贪心选择结束最早的活动，得到 $\{1, 4, 8, 11\}$，共 4 个

贪心算法

策略：每次选择**结束时间最早**且与已选活动不冲突的活动。

算法：

```
将活动按结束时间 f_i 排序
S ← ∅（已选活动集合）
last_finish ← 0（上一个选中活动的结束时间）
for i = 1 to n do
    if s_i ≥ last_finish then
        S ← S ∪ {i}
        last_finish ← f_i
    end if
end for
return S
```

时间复杂度：$O(n \log n)$（排序）$+ O(n)$（遍历）$= O(n \log n)$

手算过程

按 f_i 排序后，逐个考虑：

考虑	s_i	f_i	$last_finish$	决策
活动 1	1	4	0	$1 \geq 0$ 选！$last_finish \leftarrow 4$
活动 2	3	5	4	$3 < 4$ 冲突，跳过
活动 3	0	6	4	$0 < 4$ 冲突，跳过
活动 4	5	7	4	$5 \geq 4$ 选！$last_finish \leftarrow 7$
活动 5	3	9	7	$3 < 7$ 冲突，跳过
活动 6	5	9	7	$5 < 7$ 冲突，跳过
活动 7	6	10	7	$6 < 7$ 冲突，跳过
活动 8	8	11	7	$8 \geq 7$ 选！$last_finish \leftarrow 11$
活动 9	8	12	11	$8 < 11$ 冲突，跳过
活动 10	2	14	11	$2 < 11$ 冲突，跳过
活动 11	12	16	11	$12 \geq 11$ 选！$last_finish \leftarrow 16$

结果：$S = \{1, 4, 8, 11\}$，共 4 个活动。

9.3 贪心正确性证明：交换论证

定理：贪心选择性质

定理：设 S 是活动集合，a_1 是 S 中结束时间最早的活动。则存在一个包含 a_1 的最优解。

证明（交换论证）：

设 OPT 是任意一个最优解，设 a_k 是 OPT 中**第一个**（开始最早的）活动。

Case 1：$a_k = a_1$（OPT 的第一个活动就是结束最早的）

已经包含 a_1，证毕。

Case 2：$a_k \neq a_1$

由于 a_1 是所有活动中结束最早的，有 $f_1 \leq f_k$。

构造新解：OPT' $= (\text{OPT} \setminus \{a_k\}) \cup \{a_1\}$

验证 OPT' 可行：

OPT 中 a_k 之后的活动都满足 $s_i \geq f_k$（与 a_k 不冲突）。

由于 $f_1 \leq f_k$，这些活动也满足 $s_i \geq f_k \geq f_1$（与 a_1 不冲突）。

所以 OPT' 中的活动互不冲突。

验证 OPT' 最优：

$|\text{OPT'}| = |\text{OPT}|$（只是替换，数量不变）

所以 OPT' 也是最优解，且包含 a_1。□

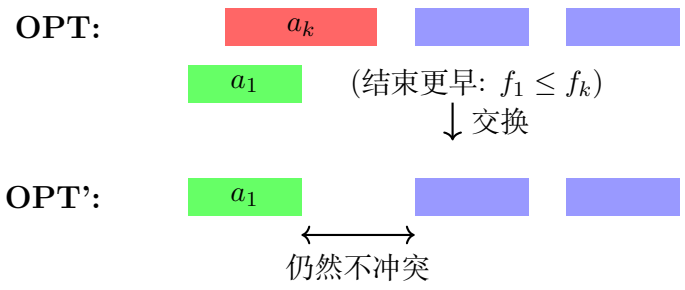


图 13: 交换论证：用 a_1 替换 a_k，后续活动仍然不冲突

定理：最优子结构

定理：设 a_1 是结束最早的活动。若 OPT 是原问题的最优解且 $a_1 \in \text{OPT}$，则 OPT $\setminus\{a_1\}$ 是子问题"a_1 结束后开始的活动" 的最优解。
子问题定义：$S' = \{a_i : s_i \ge f_1\}$（所有在 a_1 结束后才开始的活动）
证明（剪切-粘贴）：
设 OPT $\setminus\{a_1\}$ 在子问题 S' 上不是最优。
则存在 S' 的更优解 B，满足 $|B| > |\text{OPT}\setminus\{a_1\}|$。
构造 $B' = B \cup \{a_1\}$：

- B' 可行：B 中的活动都在 a_1 结束后开始，所以与 a_1 不冲突
- $|B'| = |B| + 1 > |\text{OPT}\setminus\{a_1\}| + 1 = |\text{OPT}|$

这与 OPT 是最优解矛盾！□

完整的归纳证明

定理：贪心算法得到最优解。
证明（归纳法）：
归纳假设：对于少于 n 个活动的问题，贪心得到最优解。
归纳步骤：考虑 n 个活动的问题。

1. 设 a_1 是结束最早的活动。
2. 由贪心选择性质，存在包含 a_1 的最优解 OPT。
3. 由最优子结构，OPT $\setminus\{a_1\}$ 是子问题 $S' = \{a_i : s_i \ge f_1\}$ 的最优解。
4. 子问题 S' 的活动数 $< n$，由归纳假设，贪心在 S' 上得到最优解。
5. 贪心选择 a_1，然后在 S' 上贪心，得到的解 $= \{a_1\} \cup$ (子问题最优解)。
6. 这与 OPT 大小相同，所以贪心得到最优解。□

直觉：为什么选”结束最早”而不是其他策略？

错误策略 1：选开始最早的活动
反例：活动 A=[0, 100)，B=[1, 2)，C=[3, 4)
选 A 只能选 1 个，但选 B, C 能选 2 个。
错误策略 2：选持续时间最短的活动
反例：活动 A=[0, 4)，B=[3, 6)，C=[5, 9)
B 最短（3 单位），但选 B 只能选 1 个。选 A, C 能选 2 个。
错误策略 3：选冲突最少的活动
反例：设有如下 7 个活动：

- 中心活动 $M = [3, 7)$，与 A_1, A_2, A_3 均冲突（共 3 次冲突）
- 左端活动 $A_1 = [1, 4)$，$A_2 = [2, 5)$，$A_3 = [3, 6)$，各自与 M 冲突，彼此也冲突
- 右端活动 $B_1 = [5, 8)$，$B_2 = [6, 9)$，$B_3 = [7, 10)$，各自与 M 冲突，彼此也冲突

M 共冲突 6 次，而 A_i、B_j 各仅冲突 2–3 次。按”冲突最少”策略，会优先选某个 A_i 或 B_j，这将同时排除 M 和另外 $1 \sim 2$ 个活动；而若直接选 M，虽冲突最多，但选完 M 之后左右两侧均无可选活动，只能选 1 个。
直觉上：冲突少的活动往往”挤在一堆”，选了它反而阻塞了其他互相兼容的活动；选结束最早的活动才能保证给后续留下最大余地。
为什么”结束最早”是对的？
直觉：尽早结束 $\Rightarrow$ 给后面留更多空间 $\Rightarrow$ 能选更多活动
这个直觉被交换论证严格证明了。

9.4 更多贪心经典例子

9.4.1 例：分数背包

分数背包 vs 0-1 背包

0-1 背包：物品要么完整拿，要么不拿（DP 解决）
分数背包：物品可以**拿一部分**（贪心解决！）
问题：n 个物品，第 i 个重量 w_i，价值 v_i。背包容量 W。
可以拿物品的任意比例 $0 \le x_i \le 1$。
最大化 $\sum_i x_i \cdot v_i$，约束 $\sum_i x_i \cdot w_i \le W$。
贪心策略：按**单位价值** v_i/w_i 从大到小排序，依次尽量多拿。

分数背包例子

输入：背包容量 $W = 50$

物品	重量 w_i	价值 v_i	单位价值 v_i/w_i
A	10	60	6
B	20	100	5
C	30	120	4

按单位价值排序：A (6) > B (5) > C (4)
贪心过程：

- 拿 A：全部拿，用掉容量 10，价值 60，剩余容量 40
- 拿 B：全部拿，用掉容量 20，价值 100，剩余容量 20
- 拿 C：只能拿 $20/30 = 2/3$，价值 $120 \times \frac{2}{3} = 80$

总价值：$60 + 100 + 80 = 240$

分数背包贪心正确性证明

定理：按单位价值排序的贪心策略是最优的。
证明（交换论证）：
设物品已按 $v_1/w_1 \geq v_2/w_2 \geq \cdots \geq v_n/w_n$ 排序。
设 OPT 是最优解，x_i^* 是物品 i 的取用比例。
设贪心解为 x_i^G。
假设 OPT $\neq$ 贪心解，找第一个不同的位置 k：$x_k^* < x_k^G$。
由于贪心尽量多拿前面的物品，必存在 $j > k$ 使得 $x_j^* > x_j^G$。
交换：增加 x_k，减少 x_j，保持总重量不变。
设增量为 Δ，则价值变化为：

$$\Delta \cdot v_k/w_k - \Delta \cdot v_j/w_j = \Delta \left(\frac{v_k}{w_k} - \frac{v_j}{w_j} \right) \geq 0$$

因为 $k < j$ 意味着 $v_k/w_k \geq v_j/w_j$。
所以交换后价值不减，OPT 可以逐步变成贪心解。□

9.4.2 例：跳跃游戏

跳跃游戏（LeetCode 55）

问题：数组 $A[0..n-1]$，$A[i]$ 表示从位置 i 最多能跳 $A[i]$ 步。
从位置 0 出发，能否到达位置 $n-1$?
例 1：$A = [2, 3, 1, 1, 4] \to$ **能**（0→1→4 或 0→2→3→4）
例 2：$A = [3, 2, 1, 0, 4] \to$ **不能**（卡在位置 3）
贪心策略：维护当前能到达的最远位置 $maxReach$

跳跃游戏贪心算法

```
maxReach ← 0
for i = 0 to n − 1 do
    if i > maxReach then
        return False {当前位置不可达}
    end if
    maxReach ← max(maxReach, i + A[i])
    if maxReach ≥ n − 1 then
        return True
    end if
end for
return True
```

时间复杂度：$O(n)$

跳跃游戏手算

输入：$A = [2, 3, 1, 1, 4]$

i	0	1	2	3	4
$A[i]$	2	3	1	1	4
$maxReach$	2	4	4	4	–

过程：

- $i = 0$：$0 \le maxReach = 0$ ，更新 $maxReach = \max(0, 0+2) = 2$
- $i = 1$：$1 \le 2$ ，更新 $maxReach = \max(2, 1+3) = 4 \ge 4$，**返回 True**

输入：$A = [3, 2, 1, 0, 4]$

i	0	1	2	3	4
$A[i]$	3	2	1	0	4
$maxReach$	3	3	3	3	–

过程：

- $i = 0$：$maxReach = 3$
- $i = 1$：$maxReach = \max(3, 3) = 3$
- $i = 2$：$maxReach = \max(3, 3) = 3$
- $i = 3$：$maxReach = \max(3, 3) = 3$
- $i = 4$：$4 > maxReach = 3$，**返回 False**

9.4.3 例：加油站问题

加油站问题（LeetCode 134）

问题：环形路线上有 n 个加油站。第 i 站有汽油 $gas[i]$，到下一站需要 $cost[i]$。从某站出发（油箱初始为空），能否绕一圈回到起点？若能，返回起点编号。

例：$gas = [1, 2, 3, 4, 5]$，$cost = [3, 4, 5, 1, 2]$

净收益：$[-2, -2, -2, 3, 3]$

从站 3 出发：$4-1 = 3 \to 3+5-2 = 6 \to 6+1-3 = 4 \to 4+2-4 = 2 \to 2+3-5 = 0$

答案：站 3

加油站贪心算法

关键观察：

1. 若 $\sum gas[i] < \sum cost[i]$，则无解（总油量不够）
2. 若从站 i 出发，在站 j 油量变负，则 i 到 j 之间任何站都不能作为起点

贪心策略：

```
totalTank ← 0, currTank ← 0, start ← 0
for i = 0 to n − 1 do
    totalTank ← totalTank + gas[i] − cost[i]
    currTank ← currTank + gas[i] − cost[i]
    if currTank < 0 then
        start ← i + 1 {从下一站重新开始}
        currTank ← 0
    end if
end for
if totalTank ≥ 0 then
```

```
    return start
else
    return −1
end if
```

加油站手算

输入：$gas = [1, 2, 3, 4, 5]$，$cost = [3, 4, 5, 1, 2]$

i	0	1	2	3	4
$gas[i] - cost[i]$	-2	-2	-2	3	3
$currTank$	-2→0	-2→0	-2→0	3	6
$start$	1	2	3	3	3

$totalTank = -2 - 2 - 2 + 3 + 3 = 0 \geq 0$

答案：$start = 3$

正确性：若从 0,1,2 出发都会在某处油量变负，它们之间任何站也不行（因为到达那些站时油量 ≥ 0，从那里出发只会更差），所以必须从 3 开始。

9.4.4 例：最小化最大延迟（配对型交换）

最小化最大延迟问题

问题：n 个任务，第 i 个任务需要处理时间 t_i，截止时间 d_i。

单机串行处理，最小化**最大延迟** $\max_i(f_i - d_i)$，其中 f_i 是任务 i 的完成时间。

例：3 个任务

任务	1	2	3
处理时间 t_i	3	2	1
截止时间 d_i	6	8	9

贪心策略：按截止时间 d_i 从早到晚排序（EDF: Earliest Deadline First）

最小化最大延迟手算

按 d_i 排序：任务顺序 $1 \to 2 \to 3$

计算完成时间和延迟：

任务	开始	完成 f_i	截止 d_i	延迟 $f_i - d_i$
1	0	3	6	-3
2	3	5	8	-3
3	5	6	9	-3

最大延迟：$\max(-3,-3,-3) = -3$（所有任务都提前完成）
如果用其他顺序（比如 $3 \to 2 \to 1$）：

任务	开始	完成 f_i	截止 d_i	延迟 $f_i - d_i$
3	0	1	9	-8
2	1	3	8	-5
1	3	6	6	0

最大延迟 $= 0$，比 EDF 的 -3 更差！

EDF 正确性证明（配对型交换）

定理：EDF（按截止时间排序）最小化最大延迟。
证明（交换论证）：
设 OPT 是最优调度。若 OPT 不是 EDF 顺序，则存在**逆序对**：
相邻任务 i, j 满足 i 在 j 前面执行，但 $d_i > d_j$。
交换动作：交换 i 和 j 的执行顺序。
分析交换后的延迟变化：
设交换前 i 从时刻 s 开始。

	交换前	交换后
任务 j 完成时间	$s + t_i + t_j$	$s + t_j$
任务 i 完成时间	$s + t_i$	$s + t_j + t_i$

关键观察：
交换前 j 的延迟：$(s + t_i + t_j) - d_j$
交换后 i 的延迟：$(s + t_j + t_i) - d_i$
由于 $d_i > d_j$，有：

$$(s + t_i + t_j) - d_i < (s + t_i + t_j) - d_j$$

所以交换后 i 的新延迟 $<$ 交换前 j 的延迟。
而 j 的新延迟 $(s + t_j) - d_j < (s + t_i + t_j) - d_j$（更早完成）。
结论：交换后最大延迟不增。重复消除逆序对，最终得到 EDF。□

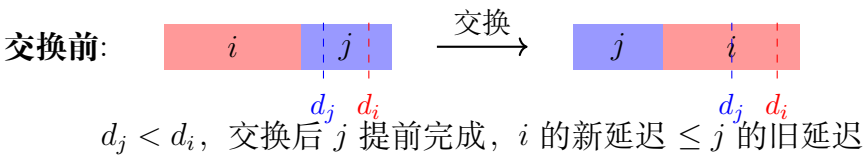


图 14: 配对型交换：交换相邻逆序对，最大延迟不增

9.4.5 例：霍夫曼编码

霍夫曼编码问题

问题：给定字符集及其频率，设计**前缀编码**使得平均编码长度最小。
前缀编码：任何字符的编码都不是另一个字符编码的前缀。
例：字符频率

字符	a	b	c	d	e	f
频率	45	13	12	16	9	5

目标：最小化 $\sum_i f_i \cdot l_i$（频率 × 编码长度）

霍夫曼算法

贪心策略：

1. 将每个字符作为单节点树，权重 = 频率
2. 重复：取权重最小的两棵树，合并成一棵新树，权重 = 两者之和
3. 直到只剩一棵树

编码：从根到叶子，左边记 0，右边记 1

霍夫曼编码手算

输入：a:45, b:13, c:12, d:16, e:9, f:5
Step 1：合并最小的 f(5) 和 e(9) → (f,e):14
剩余：a:45, d:16, (f,e):14, b:13, c:12
Step 2：合并 c(12) 和 b(13) → (c,b):25
剩余：a:45, (c,b):25, d:16, (f,e):14
Step 3：合并 (f,e)(14) 和 d(16) → ((f,e),d):30
剩余：a:45, ((f,e),d):30, (c,b):25
Step 4：合并 (c,b)(25) 和 ((f,e),d)(30) → 55
剩余：a:45, 55
Step 5：合并 a(45) 和 55 → 100（根）

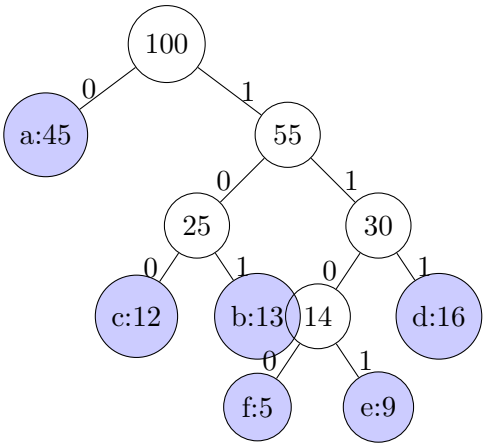


图 15: 霍夫曼树：高频字符 (a) 靠近根，低频字符 (f,e) 远离根

霍夫曼编码结果

字符	频率	编码	长度
a	45	0	1
c	12	100	3
b	13	101	3
f	5	1100	4
e	9	1101	4
d	16	111	3

平均编码长度：

$$\frac{45 \times 1 + 12 \times 3 + 13 \times 3 + 5 \times 4 + 9 \times 4 + 16 \times 3}{100} = \frac{224}{100} = 2.24$$

对比定长编码：6 个字符需要 $\lceil \log_2 6 \rceil = 3$ 位
霍夫曼节省了 $\frac{3-2.24}{3} \approx 25\%$

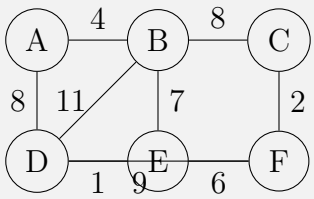
9.4.6 例：最小生成树（切割型交换）

最小生成树问题

问题：给定连通无向图 $G = (V, E)$，边权 $w : E \to \mathbb{R}^+$。
求边权和最小的生成树（连接所有顶点的无环子图）。
两个经典贪心算法：
Kruskal：按边权从小到大排序，依次加入不形成环的边
Prim：从任意顶点出发，每次加入连接树与非树顶点的最小边

Kruskal 算法例子

输入：6 个顶点，9 条边



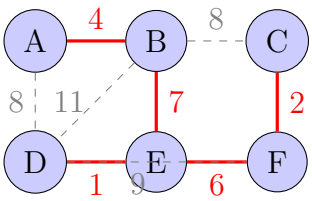
边按权重排序：(D,E):1，(C,F):2，(A,B):4，(E,F):6，(B,E):7，(A,D):8，(B,C):8，(D,F):9, (B,D):11

Kruskal 手算过程

步骤	边	权重	决策
1	(D,E)	1	加入（D 和 E 在不同组件）
2	(C,F)	2	加入（C 和 F 在不同组件）
3	(A,B)	4	加入（A 和 B 在不同组件）
4	(E,F)	6	加入（连接 {D,E} 和 {C,F}）
5	(B,E)	7	加入（连接 {A,B} 和 {C,D,E,F}）
6	(A,D)	8	跳过（A 和 D 已连通）
7	(B,C)	8	跳过（B 和 C 已连通）

MST 边集：{(D,E), (C,F), (A,B), (E,F), (B,E)}

总权重：$1 + 2 + 4 + 6 + 7 = 20$



MST 总权重 = 20

图 16: Kruskal 算法得到的最小生成树（红色粗边）

Cut Property（割性质）

定义：图 G 的一个**割** $(S, V \setminus S)$ 是顶点集的一个划分。

跨割边：一端在 S，一端在 $V \setminus S$ 的边。

Cut Property：设 $(S, V \setminus S)$ 是任意割，e 是跨割边中权重最小的（唯一最小）。则 e 属于某棵 MST。

Cut Property 证明（切割型交换）

证明：
设 T^* 是任意 MST。
Case 1：$e \in T^*$，证毕。
Case 2：$e \notin T^*$。
将 e 加入 T^* 会形成唯一的环 C。
环 C 必然还包含另一条跨割边 e'（因为 e 跨割，环必须" 回来"）。
由假设，$w(e) < w(e')$。
交换：$T' = T^* \cup \{e\} \setminus \{e'\}$
验证：

- T' 仍连通：删 e' 断开的两部分被 e 重新连接
- T' 无环：删除了环 C 中的一条边
- $w(T') = w(T^*) - w(e') + w(e) < w(T^*)$

与 T^* 是 MST 矛盾！所以 Case 2 不可能。□

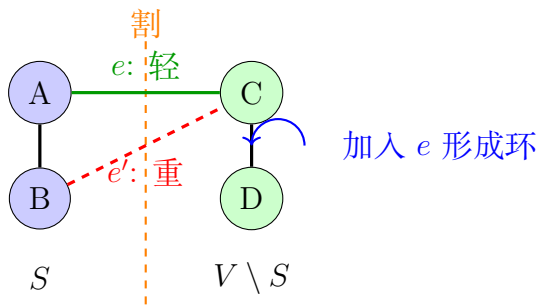


图 17: Cut Property：最小跨割边 e 必在 MST 中，否则可以替换 e'

Kruskal 正确性（基于 Cut Property）

为什么 Kruskal 正确？
当 Kruskal 选择边 $e = (u, v)$ 时：

- u 和 v 在不同的连通分量（否则会形成环）
- 设 u 所在分量为 S，则 $(S, V \setminus S)$ 是一个割

- e 是**目前未考虑的边中**权重最小的跨割边
- 由 Cut Property，e 属于某棵 MST

归纳：每一步选的边都在某棵 MST 中，最终得到 MST。

核心结论：贪心交换论证的四种类型总结

A. 排序型（活动选择、区间调度）

- 交换对象：第一个被选的元素
- 关键：贪心选的”结束更早/更小” $\Rightarrow$ 不减少后续空间

B. 配对型（最小化最大延迟、任务调度）

- 交换对象：相邻逆序对
- 关键：消除逆序后目标函数不增

C. 切割型（MST 的 Cut Property）

- 交换对象：跨割的一条”重边”
- 关键：换成最轻边后仍是生成树，且更优

D. 资源分配型（分数背包、霍夫曼编码）

- 交换对象：资源份额或编码深度
- 关键：把”更该优先的”往前挪，目标单调改进

9.5 贪心为什么难并行？

GPU 视角：贪心的顺序依赖

问题：贪心的每一步都依赖前一步的结果！
活动选择的贪心过程：

1. 选结束最早的活动 a_1
2. 在 a_1 结束后开始的活动中，选结束最早的 a_2
3. 在 a_2 结束后开始的活动中，选结束最早的 a_3
4. $\cdots$

每一步的**候选集**取决于前一步的选择，无法并行！
并行化思路：

- 预处理可以并行（如排序）
- 但核心的贪心选择过程很难并行
- 有时可以用”并行暴力”替代贪心

10 暴力的逆袭：当并行改变一切

10.1 核心思想

直觉：复杂度的重新审视

传统算法分析假设**顺序执行**：$O(n^2)$ 比 $O(n)$ 慢。
但在 GPU 上，这个关系可能**颠倒**！
关键公式：

$$\text{实际时间} = \frac{\text{总工作量}}{\text{并行度}}$$

- 聪明算法 $O(n)$，但只能串行 $\Rightarrow$ 时间 $O(n)$
- 笨算法 $O(n^2)$，但完全并行 $\Rightarrow$ 时间 $O(n^2/p)$

当 $p > n$ 时，$O(n^2/p) < O(n)$，暴力更快！

核心结论：什么时候暴力能赢

暴力更快的条件：

1. 问题规模较小（暴力的总工作量不太大）
2. 暴力算法完全并行（每个实例独立）
3. 聪明算法有串行依赖（无法充分利用 GPU）
4. GPU 有足够多的核心（并行度 > 问题规模）

实际应用：

- 小规模子问题：用 GPU 暴力
- 大规模问题：用传统算法或混合策略

11 总结与实验

核心结论：本周要点

1. **分治**：子问题独立 ⇒ 天然并行
   - Merge Sort：层级并行
   - FFT：蝴蝶操作并行
2. **动态规划**：有依赖但可以找并行结构
   - Wavefront 并行：沿对角线推进
   - 加速比可达 $O(\min(m, n))$
3. **贪心**：顺序依赖最强，最难并行
   - 预处理可以并行
   - 核心贪心选择通常串行
4. **暴力的逆袭**：当并行度足够高
   - $O(n^2)$ 并行可能快于 $O(n)$ 串行
   - 关键是找到临界点

A 附录：CUDA Kernel 示例

Listing 1: 并行合并的 CUDA Kernel

```
__global__ void parallel_merge_kernel(
    float* A, float* B, float* C,
    int m, int n
) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int total = m + n;

    if (tid >= total) return;

    // 二分搜索找到正确的分割点
    int low = max(0, tid - n);
    int high = min(tid, m);

```

```
    while (low < high) {
        int mid = (low + high) / 2;
        int j = tid - mid;

        if (j > 0 && mid < m && B[j-1] > A[mid]) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }

    int i = low;
    int j = tid - i;

    // 确定C[tid]的值
    if (i >= m) {
        C[tid] = B[j];
    } else if (j >= n) {
        C[tid] = A[i];
    } else {
        C[tid] = (A[i] <= B[j]) ? A[i] : B[j];
    }
}
```